



## 저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

박사학위논문

CCTV 기반 산업안전 관제 시스템에  
관한 연구

A Study on CCTV-based Occupational Safety  
Control System

2023년 12월

배재대학교 대학원

컴퓨터공학과 컴퓨터공학전공

류수영

# CCTV 기반 산업안전 관제 시스템에 관한 연구

지도교수 정 회 경

배재대학교 대학원

컴퓨터공학과 컴퓨터공학전공

류 수 영

2023년 12월

류수영의 박사학위논문을 제출함.

류수영의 박사학위논문을 인준함.

심사위원장

이승의



심사위원

이성욱



심사위원

최종근



심사위원

송종철



심사위원

정호정



2023년 12월

배재대학교 대학원

# CCTV 기반 산업안전 관제 시스템에 관한 연구

류 수 영

지도교수 정 회 경

배재대학교 대학원 컴퓨터공학과

중대재해처벌법 시행 이후에도 여전히 산업재해는 발생하고 있으며 이를 감소하기 위해 정부나 산업체에서 다양한 노력이 이뤄지고 있다. 특히 고용노동부와 산업안전보건공단이 발간한 2023년 고위험요인(Serious Injury and Fatality, SIF) 데이터를 보면 최근 6년간 사고 사망사례 4,432건에 대해 분야별로 건설업과 제조업이 가장 많이 차지했고 그 중 제조업에서 재해 유발 요인으로 74개의 분류 유형, 고위험 작업에 대해서는 29건에 해당하는 수치를 발표했다. 이 유형에서는 적재 상태 불량, 전원 차단, 작동된 설비에 끼임 등이 나타났다.

이러한 산업재해를 예방하기 위해서는 위험성 감소대책이 강도 높게 사업장에서 요구되며 이를 담당하는 안전관리 담당자를 두도록 법제화하고 있다. 하지만 담당자를 통한 모든 위험 요인에 대해 분석하기는 쉬운 일이 아니다. 이러한 점을 보완하기 위해 사업장에 설치된 CCTV를 활용해 관리 감독하고 이 데이터를 보관하여 이상행동 탐지에 활용하고 있다. 한국지능정보사회진흥원에서도 산업안전과 관련한 다양한 데이터를 제공하고 있다. 이상행동 탐지에는 객체 탐지, 사람의 관절 데이터인 키포인트(key point) 추출 등을 활용하고 있다. 객체 탐지에는 SSD, YOLO 등의 모델이 많이 활용되고 있으며 키포인트 추출에는

OpenPose, HRNet, AlphaPose 등의 모델이 활용되고 있다.

본 논문에서는 사업장 3곳의 CCTV에서 획득한 영상을 대상으로 시나리오를 선정하고 이에 대한 객체를 추출하였다. 객체는 작업자, 위험 구역에 안전모와 안전화와 같은 안전 장구, 위생 장구 등을 추가하였으며 시나리오로는 3개 사업장 공통으로 위험한 상황이거나 규칙을 위반한 상황인 자재 물품 적재, 안전 장구나 위생 장구의 미착용, 지게차와 같은 이동 객체의 충돌 등으로 선정하였다.

이러한 시나리오와 객체를 인식하기 위해서는 객체 탐지 모델이 필수적이며 문헌 연구와 실험을 통해 최종 YOLOv4를 선정하였다. YOLOv4의 하이퍼파라미터를 활용해 객체 검출 성능은 99%를 보였으며 시나리오에 대한 상황인식은 실외 환경 작업자 검출 과정에서 가장 낮은 값은 65.61%, 높은 값은 100%의 결과를 보였다.

이후 관제시스템을 구축하기 위해 기능을 추출하고 모듈별로 설계하였다. 관제시스템이 객체 검출과 위험판정이나 규칙 위반 알람까지 이를 모두 수행하려면 기존의 설계한 관제시스템으로는 속도 측면에서 문제가 있으며 이를 방지하고자 Python 코드 실행시간 최적화 기법, 리스트 열거의 사용, 문자열 병합 속도 최적화, 멀티프로세싱 기법 등을 활용하였다.

본 논문에서 연구한 것을 토대로 다양한 센서 및 네트워크를 구축하여 산업안전 재해를 방지하는 시스템은 물론, 이러한 시스템이 더욱 강건해지기 위한 대용량 데이터 수집에도 활용될 수 있으며, 작업자의 키포인트 추출을 통한 성능향상을 통한 개발 등에 활용될 것으로 사료된다.

주요어 : Dangerous scenario, Industrial safety, Object Detection, Monitoring system, YOLOv4

## 목 차

|                                       |           |
|---------------------------------------|-----------|
| 국문초록 .....                            | i         |
| 목 차 .....                             | iii       |
| 그림목차 .....                            | v         |
| 도표목차 .....                            | viii      |
| <br>                                  |           |
| <b>I. 서 론 .....</b>                   | <b>1</b>  |
| 1.1 연구 배경 및 목적 .....                  | 1         |
| 1.2 연구 내용 및 범위 .....                  | 3         |
| 1.3 논문의 구성 .....                      | 4         |
| <br>                                  |           |
| <b>II. 관련 연구 .....</b>                | <b>5</b>  |
| 2.1 사업장 현황분석 .....                    | 5         |
| 2.1.1 산업별, 재해유형별 현황 분포 .....          | 5         |
| 2.1.2 사업장별 현황 .....                   | 7         |
| 2.2 객체 인식 모델과 저작도구 .....              | 9         |
| 2.2.1 문헌 연구를 통한 객체 인식 모델 정확도 비교 ..... | 9         |
| 2.2.2 인공지능 저작도구 비교 .....              | 14        |
| <br>                                  |           |
| <b>III. 시험 환경과 데이터 구축 .....</b>       | <b>18</b> |
| 3.1 저작도구 선정 .....                     | 18        |
| 3.2 시험환경 위험 시나리오 선정 .....             | 19        |
| 3.3 영상데이터 수집과 전처리 .....               | 23        |
| <br>                                  |           |
| <b>IV. 시스템 설계 및 구현 .....</b>          | <b>33</b> |
| 4.1 모델 비교 실험 .....                    | 33        |
| 4.2 위험판정 알고리즘 설계 .....                | 36        |

|                              |               |
|------------------------------|---------------|
| 4.3 위험판정 성능 측정과 결과 .....     | 42            |
| 4.4 관제시스템 설계 .....           | 49            |
| 4.5 시스템 구현 결과 .....          | 64            |
| 4.6 고찰 .....                 | 69            |
| <br><b>V. 결 론</b> .....      | <br><b>71</b> |
| <br>참고문헌 .....               | <br>72        |
| 영문초록 .....                   | 80            |
| 감사의 글(Acknowledgement) ..... | 83            |

## 그 립 목 차

|                                                                                                                                      |    |
|--------------------------------------------------------------------------------------------------------------------------------------|----|
| 그림 1. Distribution of disasters by industry .....                                                                                    | 5  |
| 그림 2. Distribution of victims by disaster type .....                                                                                 | 6  |
| 그림 3. Workplace A(left) and B(right) where CCTV installations were<br>photographed .....                                             | 8  |
| 그림 4. Workplace C where CCTV installations were photographed .....                                                                   | 9  |
| 그림 5. Graph comparing the accuracy of object detection models .....                                                                  | 13 |
| 그림 6. Graph comparing the frame per second of object recognition models<br>.....                                                     | 14 |
| 그림 7. Screenshot image of CVAT in action .....                                                                                       | 15 |
| 그림 8. Screenshot image of YOLO Mark in action .....                                                                                  | 16 |
| 그림 9. Screenshot image of DarkLabel in action .....                                                                                  | 17 |
| 그림 10. Relating objects to scenarios .....                                                                                           | 23 |
| 그림 11. The process of labeling data extracted over 5 days as CVAT ..                                                                 | 24 |
| 그림 12. Inspect hygiene equipment in sanitized area upon entry and exit                                                               | 27 |
| 그림 13. Inspect for safety gear in hazardous area upon entry and exit ·                                                               | 27 |
| 그림 14. Inspect for collision with moving objects .....                                                                               | 28 |
| 그림 15. Detect more than one person working in hazardous area(left) and<br>detect when items are loaded in an aisle area(right) ..... | 28 |
| 그림 16. Photographing the status of additional CCTVs installed .....                                                                  | 29 |
| 그림 17. Examples of additional footage .....                                                                                          | 30 |
| 그림 18. Some of the environments and code built to model SSD and<br>YOLOv4 .....                                                      | 34 |
| 그림 19. Learning outcomes with Tensorflow .....                                                                                       | 35 |
| 그림 20. Check out the learning graph using Tensorboard .....                                                                          | 35 |
| 그림 21. Object detection accuracy using test data .....                                                                               | 36 |
| 그림 22. Moving object collision risk determination algorithm .....                                                                    | 37 |

|                                                                                                                               |    |
|-------------------------------------------------------------------------------------------------------------------------------|----|
| 그림 23. Results detected before and after the crash .....                                                                      | 38 |
| 그림 24. Algorithm for determining the direction of movement of the<br>previous frame object and the current frame object ..... | 38 |
| 그림 25. Collision detection algorithm and algorithm to determine if an<br>object has disappeared .....                         | 39 |
| 그림 26. Object preprocessing code for calculating similarity .....                                                             | 40 |
| 그림 27. Similarity verification code with OpenCV .....                                                                         | 40 |
| 그림 28. Histogram similarity features provided by OpenCV .....                                                                 | 42 |
| 그림 29. Performance measurement flowchart using the k-fold cross<br>validation method .....                                    | 43 |
| 그림 30. Object detection-based contextual judgment performance graph                                                           | 44 |
| 그림 31. Object detection latency calculation result graph .....                                                                | 46 |
| 그림 32. Adjust the detection IoU value to determine performance .....                                                          | 47 |
| 그림 33. A integrated design flowchart for an industrial accident prevention<br>system .....                                    | 49 |
| 그림 34. Control system hardware and network diagram .....                                                                      | 54 |
| 그림 35. Flowchart of how the modules in the control system work 1 ...                                                          | 56 |
| 그림 36. Flowchart of how the modules in the control system work 2 ...                                                          | 56 |
| 그림 37. Flowchart of how the modules in the control system work 3 ...                                                          | 57 |
| 그림 38. Flow diagram of the control system main CCTV .....                                                                     | 58 |
| 그림 39. Flow diagram of control system object detection and hazardous<br>situation judgment .....                              | 59 |
| 그림 40. Flowchart for the VideoGet module .....                                                                                | 60 |
| 그림 41. Performance comparison graph between PyPy and Python .....                                                             | 61 |
| 그림 42. Before applying list comprehension .....                                                                               | 62 |
| 그림 43. After applying list comprehension .....                                                                                | 62 |
| 그림 44. Before applying string concatenation optimization .....                                                                | 62 |
| 그림 45. After applying string concatenation optimization .....                                                                 | 63 |

|                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------|----|
| 그림 46. CCTV multi-processing creating diagram .....                                                        | 63 |
| 그림 47. Real-time monitoring and object detection and tracking display 1 ..<br>.....                        | 64 |
| 그림 48. Real-time monitoring and object detection and tracking display 2 ..<br>.....                        | 65 |
| 그림 49. Browse historical footage and view past dangerous events .....                                      | 65 |
| 그림 50. Display dangerous situations based on object detection results 1                                    | 66 |
| 그림 51. Display dangerous situations based on object detection results 2                                    | 66 |
| 그림 52. Generating audio signals for safety diagnostic status and output<br>warning light indications ..... | 67 |
| 그림 53. Login screens for managing business and users 1 .....                                               | 67 |
| 그림 54. Login screens for managing business and users 2 .....                                               | 68 |
| 그림 55. Established a real-time control system at the demonstration site<br>1 .....                         | 68 |
| 그림 56. Established a real-time control system at the demonstration site<br>2 .....                         | 69 |

## 도 표 목 차

|                                                                                                         |    |
|---------------------------------------------------------------------------------------------------------|----|
| 표 1. Select a demonstration sites .....                                                                 | 6  |
| 표 2. Features and CCTV installation status of each workplace .....                                      | 8  |
| 표 3. Table organized by feature for annotation tools .....                                              | 18 |
| 표 4. Hardware specifications of the video training data building server ·                               | 19 |
| 표 5. Typing labeling objects .....                                                                      | 20 |
| 표 6. 8 extracted scenarios .....                                                                        | 21 |
| 표 7. Number of time zones and deployment data extracted from site A                                     | 24 |
| 표 8. Shooting conditions in situations where there is a risk of collision<br>with a moving object ..... | 25 |
| 표 9. Plots and behaviors for moving objects and collision risk situations                               | 26 |
| 표 10. Total number of frames of collected video data organized by<br>scenario .....                     | 31 |
| 표 11. Schema organized in the form of a JSON file .....                                                 | 32 |
| 표 12. The environment trained for the selected model comparison<br>experiment .....                     | 33 |
| 표 13. Object detection-based contextual judgment performance results                                    | 44 |
| 표 14. Object detection latency calculation results .....                                                | 45 |
| 표 15. The result of calculating the object detection accuracy .....                                     | 48 |
| 표 16. Environment where the control system is installed .....                                           | 50 |
| 표 17. Software and libraries utilized in the control system .....                                       | 50 |
| 표 18. IP camera specifications .....                                                                    | 52 |

# I. 서 론

## 1.1 연구 배경 및 목적

인공지능, IoT, 그리고 빅데이터 기술 분야에서는 급속한 발전으로 혁신적인 제품과 서비스가 끊임없이 등장하고 소멸하는, 매우 역동적인 시장 환경이 형성되고 있다. 이러한 기술들은 나날이 산업현장에서 활용되고 있으며 개인의 건강한 삶을 보장하는 분야에서도 활발히 활용되고 있다. IoT를 통한 데이터수집을 비롯해 이러한 데이터를 분석하고 이를 예측하고 분류하는 모델이 많이 서비스되고 있다.

안전사고 예방 분야에서는 위험 인자를 분석하고 이를 분류화하여 화재, 가스, 방범, 유해 물질 인자와 같은 위험 요소, 시설물 등을 체계적으로 모니터링하고 있다. 하지만 중대재해처벌법 시행 이후에도 여전히 안전사고는 발생하고 있으며 이에 대한 대책이 산업계에 강력히 요구되고 있다. 특히 산업안전사고 중 제조, 건설 현장에서 발생하는 협착 및 추락 사고의 경우 근로자에게 매우 치명적인 안전사고 유형으로 고용노동부와 산업안전보건공단이 분석한 고위험요인(Serious Injury and Fatality, SIF) 데이터에서 분류하고 있어 작업자와 관리자들이 가장 우선으로 해결해야 하는 문제로 인식된다는 점이다. 이를 통해 해결책이 시급한 실정이다[1].

산업안전사고 예방시스템에 대한 산업체의 요구가 절실한 실정에서 제조업과 건설업에서 발생하는 치명적인 사고를 신속하게 감지하고 예방하기 위한 분석, 예측이 필요하다. 이에 관한 안전관리 기술개발을 위해 산업안전 지능형 플랫폼 기술개발로써 생활 안전까지 범위를 넓힐 수 있도록 체계적인 지능형 플랫폼 구축까지 연결되어야 한다.

본 논문에서는 산업안전 지능형 플랫폼 개발을 위한 작업장 내 네트워크를 중심으로 위험 상황을 감지하는 CCTV로 영상데이터를 활용하여 이를 분석하고 서비스로 확장할 수 있도록 할 것이다. 이를 위해서는 데이터 수집을 위한 사업장 선정이 필수적이고 이에 맞는 전처리를 수행해야 한다. 사업장은 CCTV가 설치된 3개 기업을 선정하였으며 위생 장구나 안전 장구와 같은 정밀 영상이 필요한 재현 영상 데이터도 추가하였다.

전처리를 위한 저작도구 선정과 각 사업장에서 도출된 위험 시나리오를 도출하기 위해 8개의 위험 유형과 규칙 위반 상황 시나리오를 구성하여 시나리오별로 영상을 분류한다. 이를 위반하는 상황을 규칙 DB로 생성하여 이에 맞는 알고리즘을 설계할 것이다. 이를 통해 성능을 측정하고자 정확도를 활용하고 즉각적으로 위험 상황에 반응할 수 있도록 처리시간 및 지연시간을 측정할 것이다. 측정된 성능과 최종 결과를 고찰하여 관제 시스템이 효율적으로 구축되었는지에 관해 연구할 것이다. 아울러 객체 검출과 추적 상황, 위험 상황 검출 결과를 관리자가 볼 수 있도록 실시간 모니터링 관제시스템 구축도 함께 진행할 것이다.

본 산업안전 지능형 서비스 플랫폼 연구를 통해 산업 공간 내에서 네트워크를 중심으로 각종 센서나 디바이스를 활용해 다양한 산업안전 위험 상황을 분석하고 확장할 수 있을 것으로 사료된다.

## 1.2 연구 내용 및 범위

본 논문에서는 산업안전 관제시스템에 관한 위험판정 모델을 만들기 위해 사업장 선정부터 객체 추적 모델, 데이터셋 구축을 위한 저작도와 전처리, 위험판정 성능 측정을 분석하는 연구에 관해 기술한다.

- 사업장 현황분석

관제시스템에서 모델을 학습하기 위해 데이터 구축을 위한 사업장 선정과 사업장이 어떤 위험 요인을 가지는지 기술한다.

- 시험환경 데이터셋 구축

사업장 선정을 통한 영상데이터 수집 후 저작도구를 통한 데이터 전처리와 위험 상황 시나리오 도출에 관해 기술한다.

- 위험판정 모델 구축을 위한 실험

학습데이터 구축 이후 객체 검출 모델을 비교 실험하고 이에 따른 규칙 DB를 추출하여 알고리즘 설계에 관해 기술한다.

- 위험판정 모델 구축 성능평가와 관제시스템 구축

모델 구축 이후 안전 규칙을 준수하는지 검사하고자 상황 판정 정확도를 산출하고 실시간처리에 적합한지 지연시간, 처리량을 산출하여 이를 통한 성능평가에 관해 기술한다. 아울러 관제시스템을 설계하고 구현한 결과에 관해 기술한다.

### 1.3 논문의 구성

제2장에서는 사업장 선정을 위해 산업별 재해유형별 사업장 현황을 살펴보고 판정 모델을 위한 객체 추적 알고리즘과 학습 데이터 구축을 위한 저작도구에 관해 기술한다. 제3장에서는 사업장 선정에 공통적인 위험 상황을 확인하여 위험 상황 시나리오를 선정하고 영상 데이터 수집과 이를 위한 전처리에 관해 기술한다. 제4장에서는 객체 검출 모델을 비교 실험하여 위험판정 규칙 DB를 통한 알고리즘 설계 및 판정 모델의 성능을 평가하고 관제시스템 개발 결과를 통해 고찰할 것이다. 제5장에서는 결론 및 향후 연구 방향을 제시한다.

## II. 관련 연구

본 장에서는 실증 사업장 선정을 위해 산업별, 산업유형별 사고 원인을 살펴보고 실증 사업장별 현황에 관해 사전 조사한 내용에 관해 기술한다. 아울러 문헌 연구를 통한 객체 인식 정확도를 비교하고 인공지능 저작도구를 조사하여 비교 분석한다.

### 2.1 사업장 현황분석

#### 2.1.1 산업별, 재해유형별 현황 분포

산업안전 관제시스템 구축을 위한 데이터를 확보하고 실증 시험을 하기 위한 사업장을 선정하기 위해 한국산업안전보건공단(KOSHA)에서 발간한 산업 재해 현황분석 결과를 살펴봤다. 그림 1과 그림 2는 산업별 재해 현황 분포와 재해유형별 재해현황 분포를 그래프로 표현한 것이다[2].

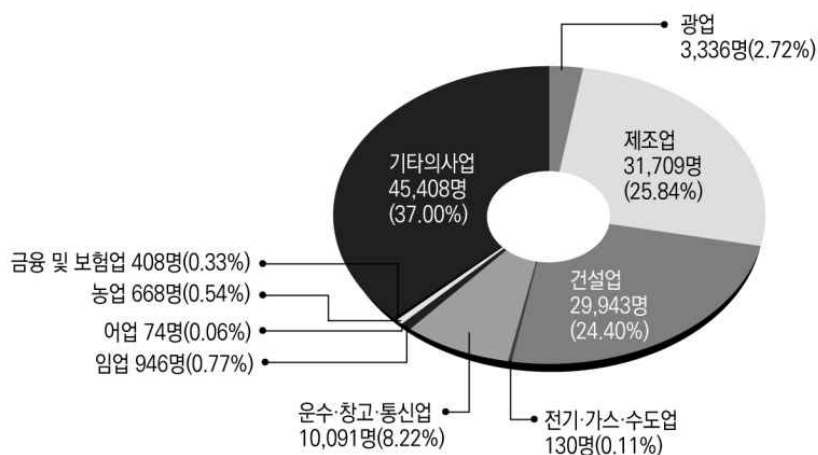


그림 1. Distribution of disasters by industry

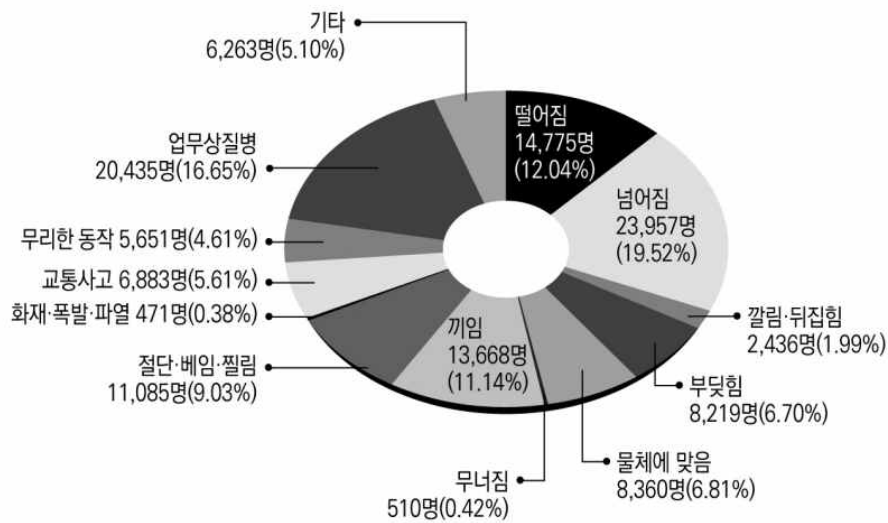


그림 2. Distribution of victims by disaster type

그림 1에서와 같이 제조업과 건설업에서 가장 많은 재해가 발생했으며, 그림 2와 같이 떨어짐, 넘어짐 등으로 사고가 가장 자주 발생하였다.

이러한 자료를 기반으로 현장 상황을 고려하여 본 논문에서는 제조업을 대상으로 선정하였고, 영상처리 기술이 활용되면서 산업현장의 상황도 함께 고려하였다. 표 1과 같이 대전광역시 내 제조 중소기업 3곳을 선정하여 진행하였다.

표 1. Select a demonstration sites

|   | Industry type | Products           | Number of workers | Location (Daejeon) |
|---|---------------|--------------------|-------------------|--------------------|
| A | Manufacturing | Automation devices | 24                | Daedeok-gu         |
| B | Manufacturing | LED Lighting       | 14                | Daedeok-gu         |
| C | Manufacturing | Mechanical parts   | 18                | Yuseong-gu         |

위 3개의 중소기업은 약 20명 내외의 근로자가 근무하고 있다. 주로 생산하는 제품의 경우 자동화기기, LED 조명, 정밀 기계 부품 등이 있어 생산하는 제품에는 조금씩 차이가 있다. 특정 재해 분야만을 선정하게 되면 모델의 범용성이 떨어질 수 있어 다양한 제품을 취급하고 생산하는 사업장으로 선정하였다.

### 2.1.2 사업장별 현황

실증 사업장 3개 기업에 대해 현황을 파악하고자 현장 방문을 통해 작업환경, 작업자 현황, 작업자가 맡고 있는 주요 업무 등을 조사하였다. 주로 작업이 이뤄지는 작업 현장으로는 생산, 제조, 가공, 조립, 포장, 자재 선적 등이 있었으며 이에 대한 위치와 구조, 배치, 동선, CCTV 설치 위치와 개수도 함께 파악하였다. 아래 표 2는 사업장별 특징과 CCTV 설치 현황을 표로 정리한 것이다.

표 2에서 정리한 바에 따르면, A 사업장에서는 정밀가공 공정에 특성을 반영하였고 적재 금지 구역에 대한 안전 구역이 있는지를 확인하였다. B 사업장에서는 위생적인 환경이 필요한 공장이었으며, 장애인이 근무하는 사업장임을 확인했다. C 사업장에서는 대형 부품을 이동하고 외부 자재 창고에서 차량이 출입할 시에 발생할 수 있는 문제를 고려하였다. 아래 그림 3은 CCTV 설치현황을 왼쪽부터 사업장 A, B를 촬영한 사진이고 그림 4는 사업장 C를 촬영한 사진이다.

표 2. Features and CCTV installation status of each workplace

| Workplace | Features                                                                                                                                                                                                                                                                                         | CCTV installation status                                                                                 |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| A         | <ul style="list-style-type: none"> <li>- Reflect the characteristics of precision machining processes such as milling and turning</li> <li>- Secure safe zones(no loading zone) inside the workplace</li> <li>- Vehicle(truck, forklift) activities such as receiving, delivery, etc.</li> </ul> | <ul style="list-style-type: none"> <li>- Indoor: 4</li> <li>- Outdoor: 8</li> <li>- Total: 12</li> </ul> |
| B         | <ul style="list-style-type: none"> <li>- Ensuring a hygienic work environment</li> <li>- Reflect the safety management characteristics of the work environment for people with disabilities</li> <li>- Vehicle(truck, forklift) activities such as warehousing, delivery, etc.</li> </ul>        | <ul style="list-style-type: none"> <li>- Indoor: 9</li> <li>- Outdoor: 7</li> <li>- Total: 16</li> </ul> |
| C         | <ul style="list-style-type: none"> <li>- Safety for moving and loading large parts</li> <li>- Managing vehicle safety at off-site warehouses</li> <li>- Vehicle(truck, forklift) activities such as receiving, delivery, etc.</li> </ul>                                                         | <ul style="list-style-type: none"> <li>- Indoor: 5</li> <li>- Outdoor: 2</li> <li>- Total: 7</li> </ul>  |



그림 3. Workplace A(left) and B(right) where CCTV installations were photographed



그림 4. Workplace C where CCTV installations were photographed

## 2.2 객체 인식 모델과 저작도구

### 2.1.1 문헌 연구를 통한 객체 인식 모델 정확도 비교

문헌 연구를 통해, 영상 객체 인식의 정확도를 비교하고자 PASCAL VOC 2007과 2012 데이터셋, 객체 인식 처리 속도를 COCO 데이터셋을 활용하였다[3-5]. 입력 이미지는 가로와 세로 크기 동일하게 288, 300, 416, 512, 544를 대상으로 하였으며 Faster R-CNN, SSD, R-FCN, YOLOv4를 활용하였다[6-14].

Faster R-CNN은 R-CNN과 Fast R-CNN을 개선한 모델로 지역 제안 네트워크(region proposal network)가 추가된 모델이다[6, 10, 11]. 이전 R-CNN의 선택적 탐색(selective search)은 객체가 있을 듯한 영역을 찾는 방법으로 초기에 객체 후보군을 여러 개 선정하여 점진적으로 통합하는 알고리즘이다[15-17]. 이는 CPU 상에서 연산하며 점진적으로 통합하는 분할 정복 알고리즘이기 때문에, 연산 속도가 매우 느리다.

Fast R-CNN에서 이를 보완하고자 선택적 탐색 알고리즘에 의해 추출된 객체 후보가 있는 지역(Region of Interest, ROI)을 하나의 CNN 네

트위크에서 수행하도록 변경하였다[18]. 이를 점진적으로 통합하는 것은 CNN의 Pooling 네트워크(ROI Pooling) 내에서 수행되므로 R-CNN보다 빠르다.

Faster R-CNN은 선택적 탐색 알고리즘을 없애 연산이 더 빠르다. 지역 제안 네트워크를 통해 객체가 어디에 있는지 예측하고자 확률을 사용한다[6]. 따라서 활성화 계층에서 Softmax 함수가 쓰인다. 클래스 개수  $j$ 가 1부터  $K$ 개 있다고 가정하고 입력이  $z$ 일 때 Softmax 함수값  $p_j$ 는 식 1과 같다.

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad (1)$$

SSD(Single Shot Detector)는 앞서 언급한 세 가지 모델과 다르다. 언급한 세 가지 모델 모두 객체나 지역을 뽑고 나서 객체를 탐지하는 2-stage 방식이다. SSD는 이를 한 번에 수행하는 모델로 YOLO(You Only Look Once) 계열의 모델도 이와 마찬가지로이다[9-11,19]. SSD는 한 번에 수행하기 위해 여러 개의 컨볼루션(convolution) 계층을 거쳐서 나온 특징 맵(feature map) 각각에 대해 객체 탐지를 수행하고 이를 통합한다[7]. 이렇게 특징 맵이 여러 개 나오기 때문에 동일한 이미지에서 동일한 객체를 여러 개 추출한다는 문제가 있어 1-stage 방식의 모델들은 비 최대치 억제(Non-Maximum Suppression, NMS) 기법을 사용해 가장 높은 확률의 객체만을 선정한다[19-21].

R-FCN은 Faster R-CNN 모델에서 사용했던 지역 제안 네트워크의 문제점을 지적하여 이를 개선한 모델이다[8]. 지역 제안 네트워크의 문제점은 모든 ROI에 대해 연산하기 때문에 속도가 느리다는 단점이 있다.

모든 ROI 중 일부는 일부 연산 값이 비슷할 수 있으며 이 값을 공유한다면 속도나 메모리 문제에서 더욱 효율적으로 된다. 공유하는 방법으로는 ROI를 격자(grid)로 나눠 위치 정보를 인코딩해서 학습한다.

YOLOv4는 YOLO의 4번째 버전으로 나온 모델이다[12, 19-23]. YOLO는 SSD와 마찬가지로 1-stage 방식이며 이미지를 격자 방식으로 나눠서 격자 하나가 클래스 확률로 변환되어 객체의 바운딩 박스(bounding box)를 추출하는 방식이다[24-28].

YOLOv2는 YOLO의 정확도가 떨어지는 문제점을 해결하기 위해 네트워크를 더 깊게 구성한 방식으로 깊게 구성하면 속도가 줄어드기 때문에, 배치 정규화(batch normalization), 고해상도 이미지를 활용한 미세 조정(fine-tuning) 등을 활용해 정확도를 높이면서 속도는 유지한 모델이다[29-34].

YOLOv3는 여러 개의 크기가 다른 이미지(multi-scale image)를 활용해 특징 맵을 추출한 방식을 사용하여 YOLOv2를 더욱 개선한 모델이며 YOLOv4는 공간 피라미드 풀링(Spatial Pyramid Pooling) 기법, PANet(Path-aggregation Network) 기법 등을 적용해 여러 개의 크기가 다른 특징 맵을 다양하게 활용하는 방법을 적용한 모델이다[35-41].

위에서 언급한 모델들을 활용해 객체 정확도를 비교하기 위해 mAP(mean Average Precision)를 사용한다. 먼저 객체 탐지에서 정밀도(precision,  $P$ )와 재현율(recall,  $R$ )을 기반으로 했을 때 정밀도는 객체를 올바르게 식별했는지를 보는 것이고, 재현율은 검출 모델이 모든 정답을 찾을 수 있는지를 보는 것이다. 이는 수식 2, 3과 같다.

$$P = \frac{TP}{TP + FP} = \frac{TP}{\text{all detections}} \quad (2)$$

$$R = \frac{TP}{TP+FN} = \frac{TP}{\text{all ground truths}} \quad (3)$$

앞서 수식에서 TP(True Positive)는 모델이 정확히 객체를 탐지한 것을 의미하고, FP(False Positive)는 존재하지 않은 객체를 모델이 객체라고 탐지했거나 기존에 존재하던 객체를 모델이 다른 객체라고 탐지한 것이다. FN(False Negative)은 모델이 객체가 있는데도 불구하고 탐지하지 못한 것을 의미한다.

한편, 객체 탐지에서는 분류모델에서 사용하는 TN(True Negative)은 사용하지 않는데, 이는 객체가 없는 것을 모델이 없다고 판단한 것으로 모델이 맞힌 것이다. 하지만 이는 탐지되지 않은 객체가 무수히 많을 수도 있으므로 이는 지표로 사용하지 않는다[42-45].

위에서 재현율을  $x$ 축, 정밀도를  $y$ 축으로 하는 그래프를 그린 것을 PR-curve라고 하며 이 곡선 아래 넓이를 구한 것을 AUC(Area Under the Curve)라고 한다. 이 넓이를 모든 점에 대해 최대 정밀도를 곱한 것이 AP(Average Precision)며 이를 표현하면 수식 4와 같으며, 클래스 개수  $N$ 으로 평균을 구한 것이 mAP이고 이는 수식 5와 같다.

$$AP_{\text{all}} = \sum_n (R_{n+1} - R_n) \cdot \max_{\tilde{R}: \tilde{R} \geq R_{n+1}} P(\tilde{R}) \quad (4)$$

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (5)$$

위에서 언급한 Faster R-CNN, SSD, R-FCN, YOLOv4의 정확도를 비교한 그래프는 그림 5와 같다. SSD와 YOLO 이름 옆에 '@' 기호와 숫자는 입력 이미지의 크기로 가로와 세로 크기가 동일한 것을 의미한다.

가령 “SSD@512” 는 SSD 모델의 입력으로 가로, 세로 크기 512인 이미지를 사용했다는 의미이다. ‘@’ 기호가 없는 모델은 가로, 세로 크기가 300인 이미지를 사용했다는 의미이다. ‘\*\*’ 가 표시된 기호는 해당 모델에서 오픈소스(open-source)로 제공하는 미세 조정된 모델에서 규모가 큰 모델을 사용했다는 의미이다. 편의상 YOLOv4를 YOLO로 표현했다.

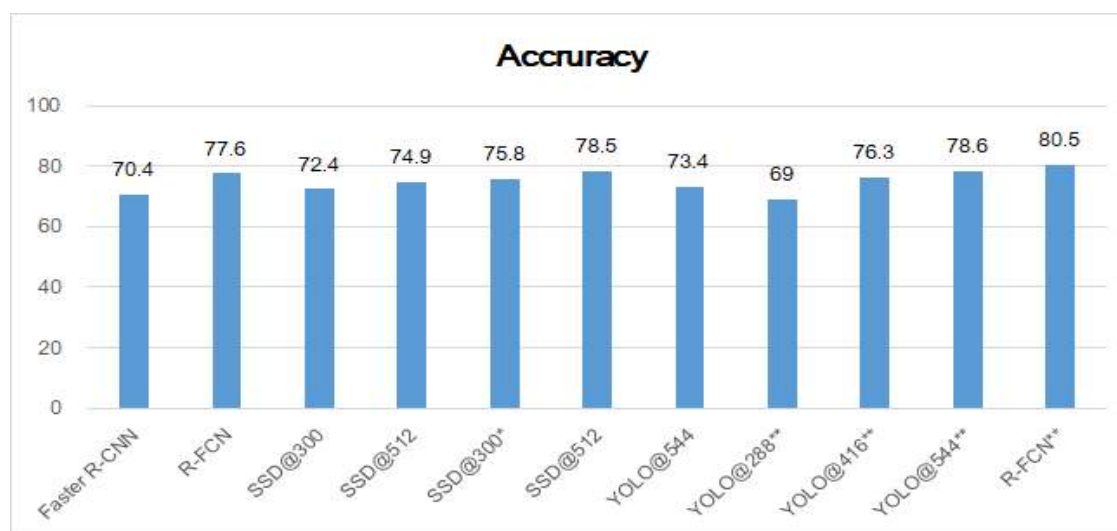


그림 5. Graph comparing the accuracy of object detection models

그림 5를 통해 R-FCN이 80.5 정확도로 가장 높은 결과를 보였으며 YOLO와 SSD 모두 비슷한 수준에 정확도를 보였다. 다만, YOLO@288 모델은 입력 이미지가 가장 작으므로 가장 낮은 정확도를 보였다.

본 논문에서는 실시간처리가 중요하므로 초당 처리 속도를 비교해야 한다. 다음 그림 6은 초당 프레임률(frame per second, fps)을 비교한 그래프이다. 모델에 명시되어 있는 (low)와 (high)는 각각 해당 모델에서 오픈소스로 제공하는 미세 조정된 모델에서 규모가 작은 모델, 큰 모델을 사용했다는 의미이다.

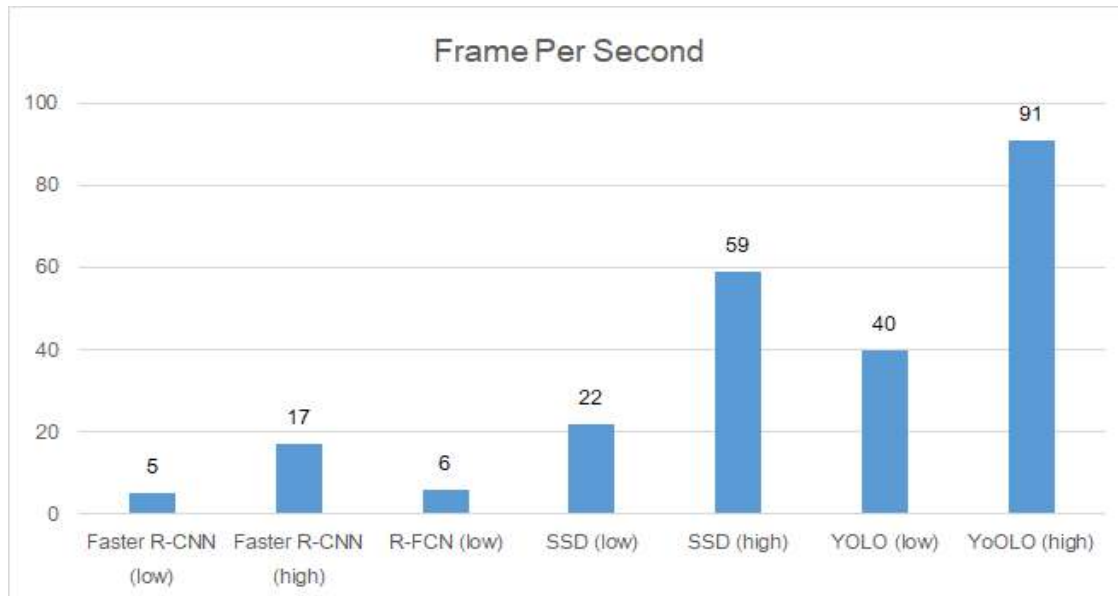


그림 6. Graph comparing the frame per second of object recognition models

### 2.2.2 인공지능 저작도구 비교

인공지능 저작도구는 인공지능 모델 학습을 위해 필요한 데이터가 모델이 이해할 수 있도록 메타데이터(metadata)와 같은 정보를 사전에 처리하는 것을 말한다. 이러한 행위를 라벨링(labeling) 혹은 어노테이션(annotation)이라 한다. 가령, 고양이 이미지가 있다면 이 이미지가 고양이 이미지이며, 고양이 이미지의  $x$ 좌푯값과  $y$ 좌푯값을 통해 어디에 있었는지 등 정보가 있을 수 있다. 이를 처리하기 위해 오픈소스로 제공하며 CVAT, YOLO Mark, Dark Label 등이 다양한 도구가 존재한다.

먼저 CVAT(Computer Vision Annotation Tool)는 Intel 연구원이 개발하여 OpenCV(Open-source Computer Vision) 재단이 배포하는 이미지, 비디오 모두 처리할 수 있는 오픈소스 저작도구이다[46-48]. 브라우저 기반이며, Google Chrome에서 작동하도록 권장하고 있다. 가상화 기반 플랫폼 Docker를 사용해 로컬 네트워크에 배포하고 이를 통해 다중

사용자가 작업이 가능한 플랫폼이다.

객체 탐지 자동 라벨링 도구도 존재해 자동으로 객체를 추적하는 기능도 제공하며 사용자 인터페이스가 단순하다는 것이 특징이다. 하지만 Docker 기반의 가상환경이기 때문에, 속도가 느리고 객체 탐지 자동 라벨링 도구를 사용하지 않으면 단순한 선형보간법에 따라 라벨링 된다는 문제가 있다. 그림 7은 CVAT가 작동하는 환경을 표현한 것이다.

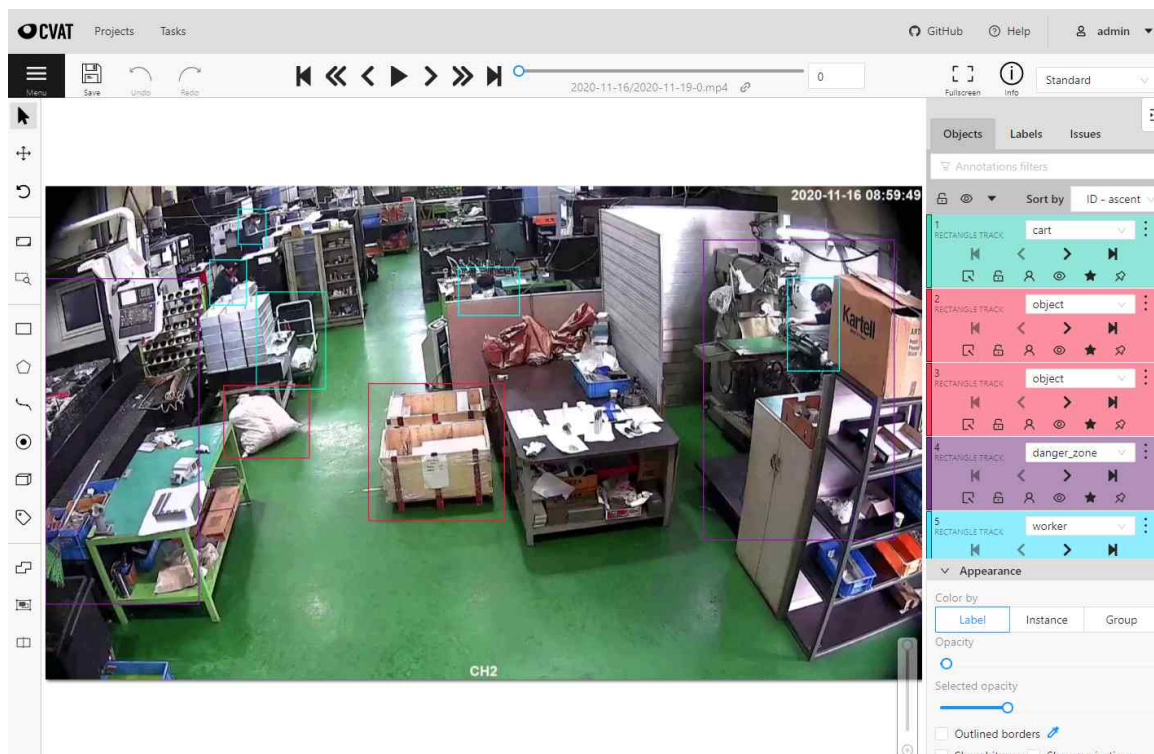


그림 7. Screenshot image of CVAT in action

YOLO Mark는 이미지, 비디오 모두 CVAT와 마찬가지로 처리할 수 있는 저작도구이다. YOLOv2 모델과 연계할 수 있으며 YOLOv2를 활용해 자동 라벨링이 가능하다는 장점이 있다[49]. 하지만 소스 코드를 직접 빌드해서 사용해야 하며, 이미지 세그멘테이션(segmentation) 기능은 존재하지 않아 바운딩 박스로만 라벨링할 수밖에 없다. 아울러 비디오 데

이터에서는 ffmpeg 라이브러리를 별도로 사용해서 프레임을 추출해야 한다는 단점이 존재한다. 아래 그림 8은 YOLO Mark가 실행되고 있는 화면을 나타낸 것이다.

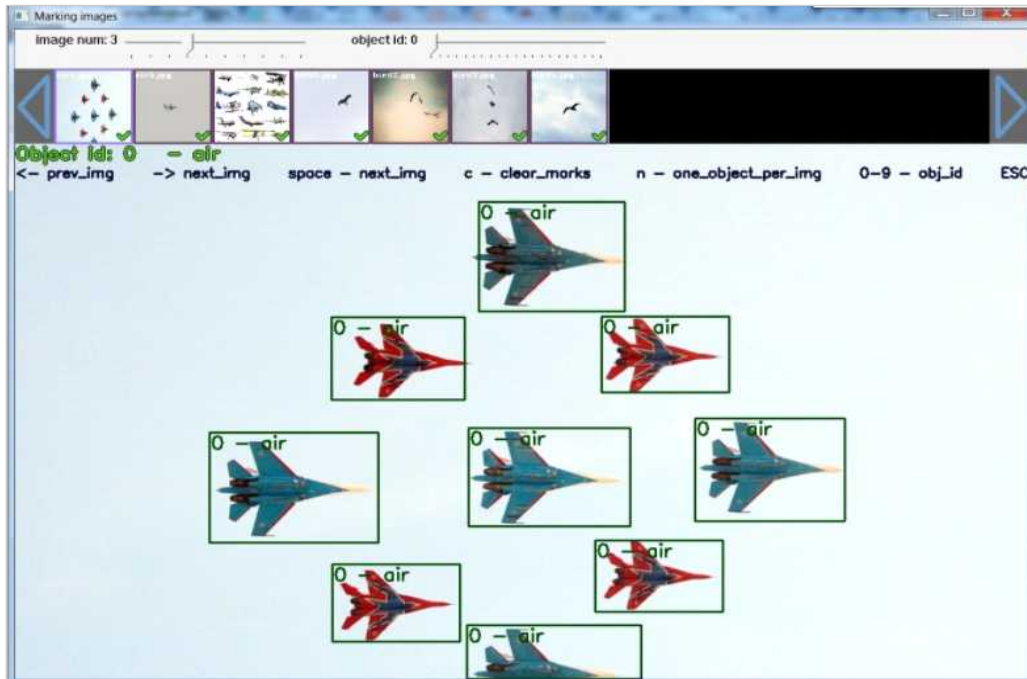


그림 8. Screenshot image of YOLO Mark in action

DarkLabel도 마찬가지로 비디오, 이미지 모두 라벨링 할 수 있으며 학습 이미지 샘플링과 모자이크 이미지 영역에 비상업적 목적으로 사용이 가능한 도구이다[50-53]. CVAT와 YOLO Mark와 다르게 다양한 데이터 포맷을 지원하며 영상에 대한 자동 프레임 추출이 가능하다. 하지만 YOLO Mark와 마찬가지로 이미지 세그멘테이션 기능을 지원하지 않고, 단순히 선형보간법에 따라 라벨링 할 수밖에 없는 단점이 있다. 아울러 Windows 환경에서만 동작한다는 단점이 있다. 그림 9는 DarkLabel이 실행된 화면을 나타낸 것이다.

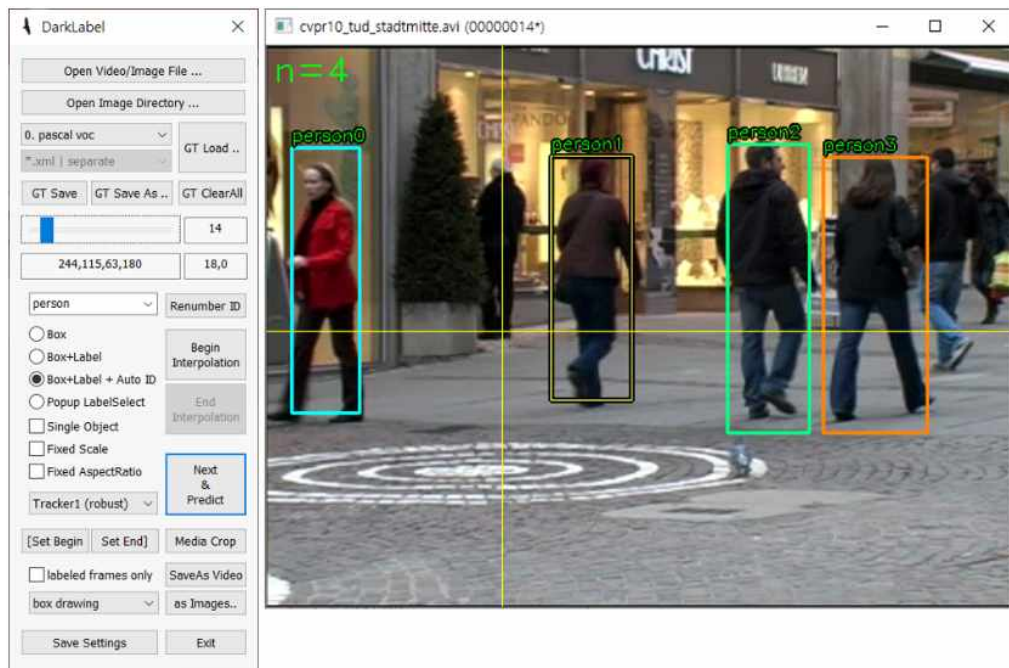


그림 9. Screenshot image of DarkLabel in action

### Ⅲ. 시험 환경과 데이터 구축

본 장에서는 저작도와 시험환경에서 위험 시나리오를 선정하고 영상 데이터를 수집한 절차와 저작도구를 활용해 최종 데이터를 전처리한 과정에 관해 기술한다.

#### 3.1 저작도구 선정

저작도구 선정은 편의성 검증을 위해 오픈소스 여부, 협업 여부가 가능한 웹 기반, 사용자 인터페이스 편의성, 자동 라벨링과 이미지 세그멘테이션 기능을 대상으로 선정하였다. 최종 CVAT로 선정하였으며 이를 통해 라벨링을 수행하였다. 저작도구에 대한 기능별로 정리한 표는 아래 표 3과 같다.

표 3. Table organized by feature for annotation tools

|                    | CVAT          | YOLO Mark     | DarkLabel          |
|--------------------|---------------|---------------|--------------------|
| Open-source        | Supported     | Supported     | Non-commercial use |
| Web-based          | Supported     | Not supported | Not supported      |
| UI                 | User-friendly |               | User-friendly      |
| Auto Labeling      | Supported     | Supported     | Not supported      |
| Image Segmentation | Supported     | Not supported | Not supported      |

영상 학습 데이터를 구축하기 위해 영상처리, 저장, 협업 기능 등을 수행하기 위해 Ubuntu 환경과 Docker를 설치해 운영하였다. 아래 표 4는 영상 학습 데이터 구축 서버의 하드웨어 사양을 표시한 것이다.

표 4. Hardware specifications of the video training data building server

|         | Content                                           |
|---------|---------------------------------------------------|
| CPU     | Intel i7-10700 (2.9Ghz, x16 core)                 |
| RAM     | 16GB                                              |
| GPU     | Nvidia Geforce GTX 1660                           |
| Storage | 250GB SSD, 4TB HDD                                |
| OS      | Ubuntu 20.04.1 LTS                                |
| Docker  | 20.10.0                                           |
| Library | Apache2-dev, Git, Python3, OpenH264 Codec, Django |

### 3.2 시험환경 위험 시나리오 선정

실증 사업장에 대한 위험 상황별 등급을 판정하기 위해 세부 시나리오를 구성해야 한다. 시나리오 구성에 앞서 연계하고자 하는 식별 객체 유형을 구분해 정의했으며, 크게 작업자, 이동 객체, 위험 및 위생 구역, 안전 및 위생 장구로 구분해 정의하였다. 아래 표 5는 라벨링하고자 하는 객체를 유형화하여 정리한 것이다.

표 5. Typing labeling objects

| No. | Object type    | Definition                                                                                 |
|-----|----------------|--------------------------------------------------------------------------------------------|
| 1   | Worker         | The person doing the work<br>(but not the person in the vehicle)                           |
| 2   | Dangerous zone | Areas that perform operations such as cutting, crimping, etc., such as machining equipment |
| 3   | Sanitary zone  | Areas that require sanitary operations, such as sanitary product production and packaging  |
| 4   | Moving object  | Mobile objects, such as carts, trucks, and forklifts                                       |
| 5   | Dangerous gear | 4 pieces of workplace safety gear<br>(hard hat, coveralls, safety shoes, gloves)           |
| 6   | Sanitary gear  | 4 pieces of workplace sanitary gear<br>(sanitary hat, coverall, gloves)                    |

선정된 실증 사업장 세 개의 지점을 기준으로 작업 현장에서 현황을 파악하기 위해 위험 및 규칙 위반 목록을 수집했으며, 이를 통해 가장 범용적인 8가지 유형의 시나리오를 최종 수집하였다.

위험 상황에 대한 정의는 위험한 행동, 안전사고가 예상되거나 산업안전을 확보하기 위해 규정된 법률로서, 혹은 가이드라인, 회사 내에서 규정된 안전 규칙을 준수하지 않은 상태를 의미한다. 이러한 상황이 발생하면 가까운 시간 내에 산업안전사고가 일어날 가능성이 높은 상황임을 의미한다.

규칙 위반에 대한 정의는 산업안전을 확보하기 위해 규정된 안전 규칙이나 생산 제품의 위생 안전도를 확보하기 위해 산업안전 가이드라인이나 법률, 회사 내에서 규정된 위생 규칙을 준수하지 않은 상태를 의미한다.

이러한 위험 상황과 규칙 위반을 바탕으로 상황, 작업자의 위험 정도, 산업안전에서 요구되는 특징, 작업장 환경, 내용, 작업장에 있는 장비나

적재 물품과 같은 구성요소 등을 종합적으로 구성하여 세부 시나리오를 구성하였다.

위험 상황과 규칙 위반 상황을 구분해 위험 상황은 현장 작업자가 이동 객체에 충돌한 위험한 상황을 기반으로 하였으며, 규칙 위반 상황의 경우 작업자들이 현장에 투입되기 직전에 위생 장구나 안전 장구를 착용했는지 판단하는 상황을 위주로 구성하였다[28]. 아래 표 6은 추출한 8가지 시나리오를 정리한 것이다.

표 6. 8 extracted scenarios

| No. | Situation      | Environment | Definition                                       | Components     |               |
|-----|----------------|-------------|--------------------------------------------------|----------------|---------------|
| 1   | Dangerous      | Indoor      | More than one person working in a dangerous area | Dangerous area | Worker        |
| 2   | Dangerous      | Indoor      | Collision risk with moving objects               | Moving objects | Worker        |
| 3   | Dangerous      | Indoor      | Loading items in the aisle area                  | Aisle area     | Loading items |
| 4   | Dangerous      | Outdoor     | Collision risk with moving objects               | Moving objects | Worker        |
| 5   | Rule violation | Indoor      | More than one person working in a sanitary area  | Sanitary area  | Worker        |
| 6   | Rule violation | Indoor      | Wear hygiene while working                       | Sanitary area  | Worker        |
| 7   | Rule violation | Indoor      | Wear safety gear                                 | Worker         | Safety gear   |
| 8   | Rule violation | Indoor      | Wear sanitary gear                               | Worker         | Sanitary gear |

표 6을 설명하면 시나리오 1은 실내 작업 현장에서 가공, 제조 장비를 가동하고 있는 위험구역 내에서 2명 이상의 불필요한 작업자가 장시간 위치하는 상황을 위험 상황으로 보는 것이다. 시나리오 2는 실내 작업 현

장에서 작업자와 물품이 적재된 카트나 대차 등 이동 객체가 서로 움직이고 있는 상황에서 작업자와 이동 객체가 충돌할 수도 있는 상황을 위험 상황으로 본 것이다.

시나리오 3은 실내 작업 현장에서 작업자가 움직일 수 있는 통로 구역 내에서 새로운 적재 물품이나 상품이 장시간 적재된 경우, 혹은 통로 구역 내에서 작업자나 이동 객체가 이동하는 중에 방해되는 경우를 위험 상황으로 본 것이다. 시나리오 4는 실외 작업 현장에서 입고나 출고 등으로 인해 작업자가 트럭이나 지게차와 같은 이동 객체에 의해 서로 충돌할 수 있는 경우를 위험 상황으로 본 것이다.

시나리오 5는 실내 작업 현장에서 가공, 제조, 시험 등이 이뤄지는 환경에서 위생 구역 내에서 2명 이상의 불필요한 작업자가 장시간 있는 경우를 규칙 위반 상황으로 본 것이다. 시나리오 6은 실내 작업 현장에서 작업자가 위생 장구나 안전 장구를 착용하지 않고 장시간 작업하는 경우를 규칙 위반 상황으로 본 것이다.

시나리오 7은 실내 작업 현장에서 작업자가 작업장 출입 시에 안전 장구를 착용하지 않고 투입되는 상황, 시나리오 8도 마찬가지로 작업장 출입 시에 위생 장구를 착용하지 않고 투입되는 상황을 모두 규칙 위반 상황으로 본 것이다. 아래 그림 10은 영상에 등장하는 객체와 시나리오에 관한 연관성을 그림으로 표현한 것이다

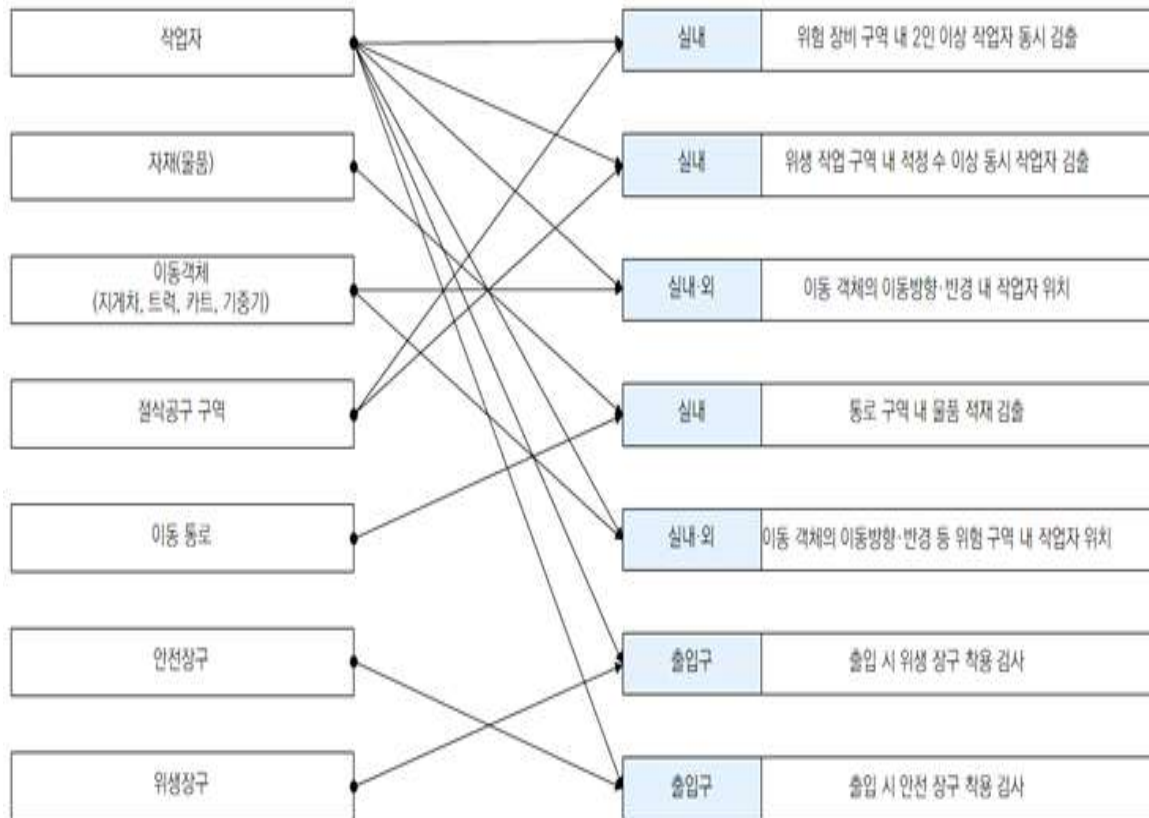


그림 10. Relating objects to scenarios

### 3.3 영상데이터 수집과 전처리

먼저 사업장 A에 설치되어 있는 CCTV를 활용해 근무 시간 기준 5일과 하루 8시간을 기준으로 40시간 데이터를 확보하였다. 이 영상은 시나리오에 따른 개수가 부족하므로 후술할 추가 데이터 영상을 확보할 것이다. 처음 확보한 데이터 가운데 매일 시간대를 달리해 2,000개 프레임을 추출하였다. 영상은 초당 25프레임이며, 이를 초당 5프레임씩 샘플링(sampling)해 추출했다. 총 10,000개의 프레임에 대해 학습데이터를 구축하였다. 아래 표 7은 추출 시간대와 해당되는 구축 데이터 수를 표현한 것이다.

표 7. Number of time zones and deployment data extracted from site A

|       | Extraction time zone | Number of frames |
|-------|----------------------|------------------|
| Day 1 | AM 09:00             | 2,000            |
| Day 2 | AM 10:00             | 2,000            |
| Day 3 | AM 11:00             | 2,000            |
| Day 4 | PM 01:00             | 2,000            |
| Day 5 | PM 02:00             | 2,000            |
| Total | 10,000               |                  |

아래 그림 11은 5일 동안 추출한 데이터에 대해 CVAT로 라벨링한 과정을 그림으로 표현한 것이다.



그림 11. The process of labeling data extracted over 5 days as CVAT

앞서 촬영한 영상에 추가하여 위험 및 규칙 위반 상황 감지에 대한 학습데이터를 추가로 확보하기 위해 시나리오에 대해 촬영조건과 플롯, 촬영계획 등을 세부적으로 계획하여 촬영을 진행하였다. 촬영조건으로는 장소나 횟수, 객체, 안전 장구와 같은 소품에 해당하고 플롯은 동작이나 행동 내용을 의미한다. 아래 표 8은 이동 객체와의 충돌위험 상황을 가정한 상황에서 촬영조건을 정리한 것이다.

표 8. Shooting conditions in situations where there is a risk of collision with a moving object

| Shooting condition | Description                                                                                                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Location           | Location 1) Outside the workplace<br>(parking lot outside the mask manufacturing plant)<br>Location 2) Inside the workplace<br>(inside walkway of the mask manufacturing plant) |
| Number of shots    | 30+ shots per location and plot                                                                                                                                                 |
| Object             | 2 workers<br>Outdoor: 1 of trucks/forklifts<br>Indoor: cart                                                                                                                     |
| Props              | Not applicable                                                                                                                                                                  |

표 8에서 이동 객체와 충돌위험 상황을 가정했을 때 촬영장소는 작업장 외부 주차장과 작업장 내부이며 촬영 횟수는 장소와 플롯 당 30회 이상 촬영을 의미하고 객체는 작업자 2명, 소품은 필요 없는 조건을 가정한 것이다. 아래 표 9는 동일한 상황에서 플롯과 그에 맞는 동작을 표로 정리한 것이다.

표 9. Plots and behaviors for moving objects and collision risk situations

| Plot | Behavior | Description                                                                 |
|------|----------|-----------------------------------------------------------------------------|
| #1   | 1        | Worker A is stationary in the workspace (backed up).                        |
|      | 2        | Moving object B is in motion.                                               |
|      | 3        | Moving object B's path of travel is toward worker A.                        |
| #2   | 1        | Workers A and B are in the workplace.                                       |
|      | 2        | Moving object C is moving.                                                  |
|      | 3        | The path of moving object B is toward one of the workers.                   |
| #3   | 1        | Moving object A is stationary and there are workers B stationary around it. |
|      | 2        | The workers remain stationary when moving object A starts moving.           |

표 9에서 이동 객체와 충돌위험 상황에 대한 플롯과 그에 따른 동작을 설명하면 플롯 1은 작업자 A가 작업 공간에서 등진 상태로 정지 상태에 있을 때 이동 객체 B가 이동 중인 상황에서 작업자 A에게 향하는 상황이다. 플롯 2는 작업자 A와 B가 함께 작업 공간에 있을 때, 이동 객체 C가 이동 중이고, 이동 객체 B가 작업자 A와 B 중 1명으로 향하는 상황이다. 플롯 3은 이동 객체 A가 정지한 상태에서 그 주위에 작업자 B가 정지한 상황에서 이동 객체 A가 이동을 시작할 때 작업자 B가 그대로 정지한 상황이다.

먼저 진/출입 시, 위생 구역 내에서 위생 장구를 착용 여부를 검사하기 위해 객체는 작업자와 위생 장구인 상황에서 작업자 A가 위생 장구 미착용한 상황에서 진출입하고 작업자 A와 B 모두 위생 장구를 미착용한 진출입한 상황은 그림 12와 같다.



그림 12. Inspect hygiene equipment in sanitized area upon entry and  
 진/출입 시, 위험구역 내에서 안전 장구를 착용 여부를 검사하기 위해  
 객체는 작업자와 안전 장구인 상황에서 작업자 A가 안전 장구 미착용한  
 상황에서 진출입하고 작업자 A와 B 모두 안전 장구를 미착용한 진출입한  
 상황이거나 위험구역 내일 때 그림 13과 같다.



그림 13. Inspect for safety gear in hazardous area upon entry and exit

이동 객체와 충돌을 검사하기 위해 객체는 작업자와 카트, 지게차를 대  
 상으로 했고 작업장 실내와 실외 모두를 검사하기 위해 작업자 A가 작업  
 도중 작업자 B에게 카트를 끌고 향하는 상황과 작업자 A가 작업하거나  
 대기 중일 때 지게차(또는 트럭)가 가까이 있는 장면을 촬영한 상황은 그  
 립 14와 같다.



그림 14. Inspect for collision with moving objects

위험구역 내에서 2인 이상 작업을 검출하기 위해 객체는 작업자이고 작업자 A가 작업 중 작업자 B가 작업 공간에 진입하거나 혹은 진입을 반복하는 상황, 작업자 A와 작업자 B가 동시에 진입하거나 대기하는 경우를 촬영한 상황은 그림 15의 왼쪽 사진이다. 통로 구역 내에서 물품이 적재된 상황을 검출하기 위해 객체는 작업자, 자재, 제품이고 작업자 A가 자재를 가지고 작업장에 진입해서 작업자 A가 작업 중 작업자 B가 주변에 자재를 적재한 후 화면에서 사라지는 상황은 그림 15의 오른쪽 사진이다.

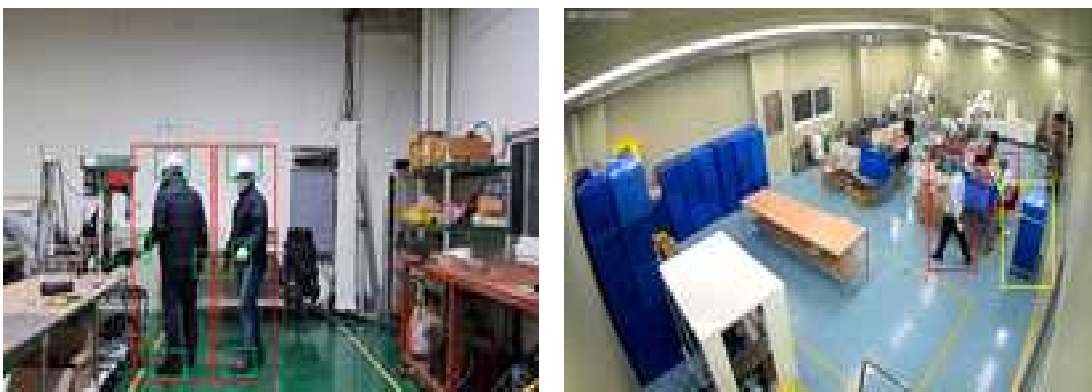


그림 15. Detect more than one person working in hazardous area(left) and detect when items are loaded in an aisle area(right)

이렇게 플롯과 행동을 설정한 추가 영상과 더불어 실증 사업장에 CCTV를 추가로 설치하였다. 실증 사업장 세 곳 모두 기존 설치되어 있던 CCTV 외에 추가 영상데이터를 더 확보하기 위해서이다. 아울러 영상 데이터 정확도 향상을 위해 기존 설치되어 있던 CCTV의 위치나 각도를 일부 조절하였다.

작업자의 위생 장구와 안전 장구 착용 여부를 파악하기 위해 작업장 출입구 정면을 향하는 위치에 CCTV를 추가 설치하였으며, 작업자와 이동 객체의 움직임을 더욱 정밀하게 파악하고자 실내 작업장 또는 실외 공간 촬영을 위해 고각도와 저각도 위치에 CCTV를 추가로 설치하였다. 아래 그림 16은 추가로 설치된 CCTV 현황을 왼쪽부터 오른쪽까지 사업장 A 부터 C까지를 차례로 촬영한 것이다.



그림 16. Photographing the status of additional CCTVs installed

진/출입 시 위생 장구와 안전 장구 착용 여부 탐지, 위생 구역 내에서 위생 장구 착용 여부 탐지의 경우 확보한 영상데이터가 현저히 적어 추가로 촬영하였다. 진/출입 시 위생 및 안전 장구 착용 여부는 출입구 정면 촬영 사진이 필요하고 다양한 위생 장구나 안전 장구가 필요하며 미착용한 작업자보다 착용한 작업자가 많아 착용하지 않은 경우도 포함되어야 하기 때문이다. 이러한 영상데이터의 예는 그림 17과 같다. 수집한 영상데이터를 시나리오별로 정리한 총 프레임 개수는 표 10과 같다.



그림 17. Examples of additional footage

표 10. Total number of frames of collected video data organized by scenario

| No.   | Scenario content                                                           | Number of frames | Location |
|-------|----------------------------------------------------------------------------|------------------|----------|
| 1     | Detect more than one worker in a hazardous equipment area at the same time | 5,849            | Indoor   |
| 2     | Detect more than the appropriate number of workers in a sanitary work area | 21,758           | Indoor   |
| 3     | Direction of movement of moving objects, location of workers in radius     | 10,312           | Indoor   |
| 4     | Detection of stacked items in the aisle area                               | 235,073          | Indoor   |
| 5     | Inspecting the wearing of hygiene equipment during work                    | 21,240           | Indoor   |
| 6     | Location of workers in the danger zone of moving objects                   | 16,705           | Outdoor  |
| 7     | Inspection of hygiene equipment                                            | 19,330           | Door     |
| 8     | Inspection of safety equipment                                             | 20,025           | Door     |
| Total |                                                                            | 350,292          |          |

아래 표 11은 해당 영상의 메타데이터, 프레임 정보, 이벤트, 객체 정보를 표기하기 위해 JSON 파일 형태로 구성된 스키마를 표시한 것이다.

## Ⅹ 11. Schema organized in the form of a JSON file

| Level 1                               | Level 2                                  | Level 3                          | Level 4                         |
|---------------------------------------|------------------------------------------|----------------------------------|---------------------------------|
| video: Original video info            | video_name:<br>file name                 |                                  |                                 |
|                                       | size: video size                         | width:<br>video width            |                                 |
|                                       |                                          | height:<br>video height          |                                 |
|                                       | duration:<br>Total duration              |                                  |                                 |
|                                       | fps: Frame per seconds                   |                                  |                                 |
|                                       | frame_length:<br>Number of frames        |                                  |                                 |
|                                       | format:<br>video format                  |                                  |                                 |
|                                       | location:<br>take location               |                                  |                                 |
|                                       | start_time:<br>video start time          |                                  |                                 |
|                                       | end_time:<br>video end time              |                                  |                                 |
| partition:<br>partition of video info | partiton_number:<br>number of partitions |                                  |                                 |
|                                       | frames:<br>partition frame info          | frame_number:<br>frame number    |                                 |
|                                       |                                          | events:<br>event info            | event_id:<br>event identifier   |
|                                       |                                          |                                  | event_name:<br>event name       |
|                                       |                                          |                                  | event_number:<br>event number   |
|                                       |                                          | objects:<br>object info in frame | object_id:<br>object identifier |
|                                       |                                          |                                  | object_name:<br>object name     |
|                                       |                                          |                                  | object_number:<br>object number |
|                                       |                                          |                                  | x: center x                     |
|                                       |                                          |                                  | y: center y                     |
|                                       |                                          |                                  | w:<br>width of object           |
|                                       |                                          |                                  | h:<br>height of object          |

## IV. 시스템 설계 및 구현

이 장에서는 본 논문에서 제안하는 시스템을 설계하기 위해 모델 비교 실험과 위험판정 알고리즘, 성능 측정과 구현 결과, 관제시스템 구축에 관해 서술한다.

### 4.1 모델 비교 실험

산업안전 관제시스템을 구축하기 위해 앞서 2장에서 설명한 SSD와 YOLOv4를 비교 실험하였다. 선정된 모델 비교 실험을 위해 학습한 시스템 환경은 표 12와 같다.

표 12. The environment trained for the selected model comparison experiment

|         | Content                                                                                       |
|---------|-----------------------------------------------------------------------------------------------|
| CPU     | Intel i7-10700 (2.9Ghz, x16 core)                                                             |
| RAM     | 16GB                                                                                          |
| GPU     | Nvidia Geforce GTX 1660                                                                       |
| Storage | 250GB SSD, 4TB HDD                                                                            |
| OS      | Ubuntu 20.04.1 LTS                                                                            |
| Library | Anaconda 3, CUDA 11.1, cuDNN 7, Data serialization, COCO API, Tensorflow object detection api |

이미지 라벨링 결과를 학습과 테스트 영상 구간으로 구분해 저장하여 7대3 비율로 지정했다. 사전 훈련된 모델을 통해 전이학습(transfer learning) 방법으로 수행했다[29]. SSD와 YOLOv4의 모델 구축을 위해 구축한 일부 환경과 코드는 그림 18과 같다.

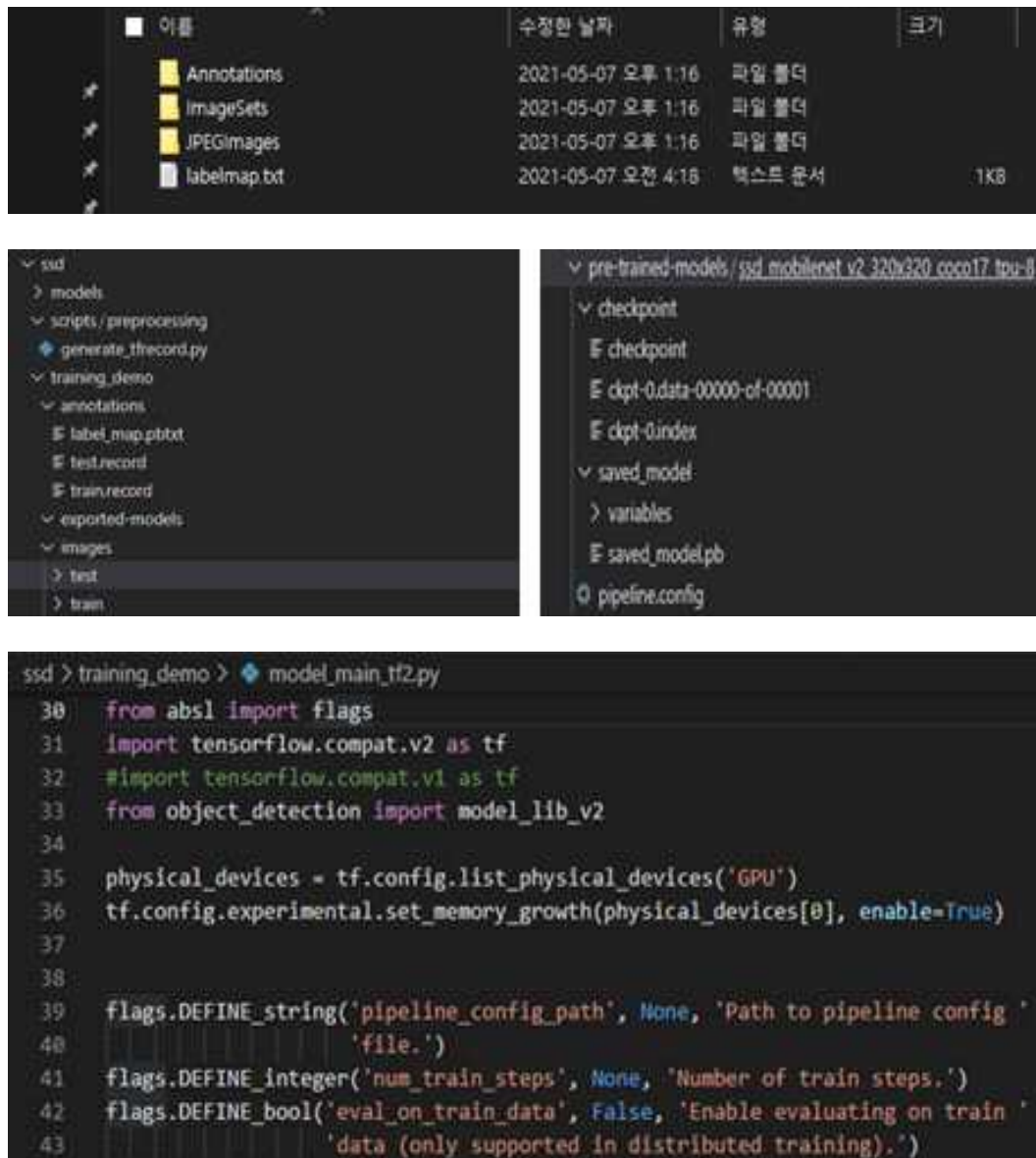


그림 18. Some of the environments and code built to model SSD and YOLOv4

데이터를 활용한 모델 학습을 수행했을 때, Tensorflow에서 제공하는 Tensorboard를 활용해 학습 결과를 확인하였으며 결과는 그림 19, 20과 같다. 그림 21은 테스트 데이터[35]를 활용해 객체 검출 정확도를 산출했을 때 결과로 객체 유형별 검출 정확도 mAP를 산출한 결과이다.

```

INFO:tensorflow:Step 100 per-step time 0.811s loss=0.478
I0508 18:48:57.530306 140092259944256 model_lib_v2.py:680] Step 100 per-step time 0.811s loss=0.478
INFO:tensorflow:Step 200 per-step time 0.516s loss=0.394
I0508 18:49:49.128457 140092259944256 model_lib_v2.py:680] Step 200 per-step time 0.516s loss=0.394
INFO:tensorflow:Step 300 per-step time 0.517s loss=0.316
I0508 18:50:40.855935 140092259944256 model_lib_v2.py:680] Step 300 per-step time 0.517s loss=0.316
INFO:tensorflow:Step 400 per-step time 0.519s loss=0.328
I0508 18:51:32.765271 140092259944256 model_lib_v2.py:680] Step 400 per-step time 0.519s loss=0.328
INFO:tensorflow:Step 500 per-step time 0.517s loss=0.385
I0508 18:52:24.439200 140092259944256 model_lib_v2.py:680] Step 500 per-step time 0.517s loss=0.385
INFO:tensorflow:Step 600 per-step time 0.516s loss=0.329
I0508 18:53:16.073879 140092259944256 model_lib_v2.py:680] Step 600 per-step time 0.516s loss=0.329
INFO:tensorflow:Step 700 per-step time 0.519s loss=0.290
I0508 18:54:07.934644 140092259944256 model_lib_v2.py:680] Step 700 per-step time 0.519s loss=0.290
INFO:tensorflow:Step 800 per-step time 0.519s loss=0.293
I0508 18:54:59.851819 140092259944256 model_lib_v2.py:680] Step 800 per-step time 0.519s loss=0.293
INFO:tensorflow:Step 900 per-step time 0.519s loss=0.294
I0508 18:55:51.760927 140092259944256 model_lib_v2.py:680] Step 900 per-step time 0.519s loss=0.294
INFO:tensorflow:Step 1000 per-step time 0.519s loss=0.324

```

그림 19. Learning outcomes with Tensorflow

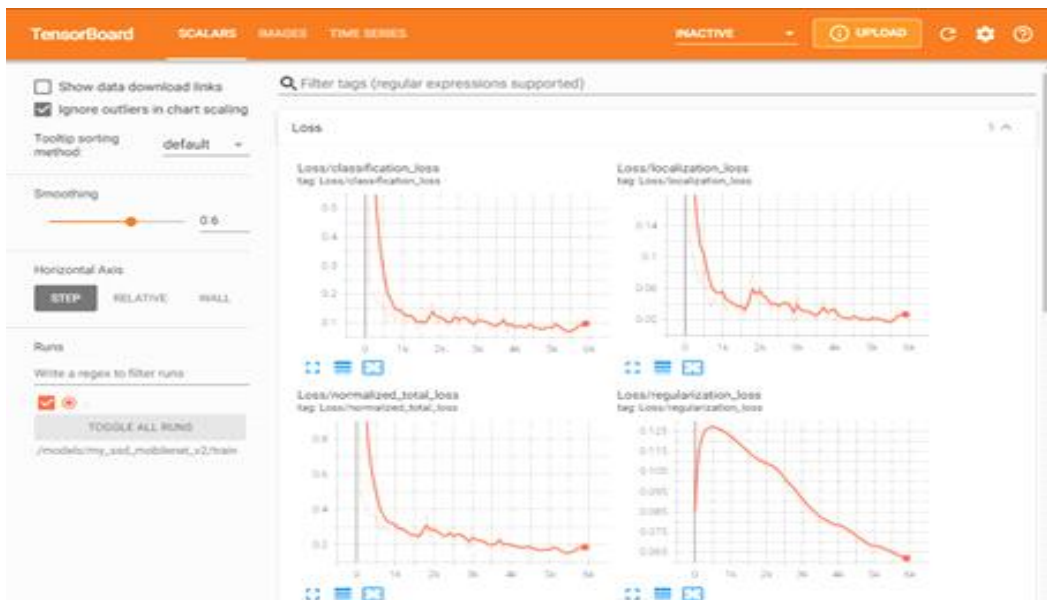


그림 20. Check out the learning graph using Tensorboard

```

class_id = 0, name = Person, ap = 0.00%      (TP = 0, FP = 0)
class_id = 1, name = Object, ap = 0.00%      (TP = 0, FP = 0)
class_id = 2, name = DangerZone, ap = 0.00%   (TP = 0, FP = 0)
class_id = 3, name = Road, ap = 0.00%         (TP = 0, FP = 0)
class_id = 4, name = worker, ap = 98.88%      (TP = 7218, FP = 304)
class_id = 5, name = object, ap = 99.96%      (TP = 9288, FP = 62)
class_id = 6, name = cart, ap = 100.00%       (TP = 3559, FP = 1)
class_id = 7, name = fork_lift, ap = 98.54%   (TP = 17, FP = 2)
class_id = 8, name = truck, ap = 0.00%        (TP = 0, FP = 0)
class_id = 9, name = sanitary_cap, ap = 0.00% (TP = 0, FP = 0)
class_id = 10, name = sanitary_clothes, ap = 0.00% (TP = 0, FP = 0)
class_id = 11, name = mask, ap = 0.00%        (TP = 0, FP = 0)
class_id = 12, name = gloves, ap = 0.00%      (TP = 0, FP = 0)
class_id = 13, name = danger_zone, ap = 100.00% (TP = 7144, FP = 2)

```

그림 21. Object detection accuracy using test data

YOLOv4와 SSD의 객체탐지 모델 결과는 YOLOv4는 0.81 mAP, SSD는 0.47 mAP가 산출되었으며 YOLOv4가 최종 선정되었다.

## 4.2 위험판정 알고리즘 설계

YOLOv4와 OpenCV를 활용하여 객체 탐지와 위험 규칙 위반을 처리하고자 한다. 모델 구성을 통해 YOLOv4의 하이퍼파라미터를 일부 조정하여 검출 및 처리 성능이 향상되었다. 특히, 이동 객체 충돌위험 판정 알고리즘의 경우 이에 관한 흐름도는 그림 22와 같다.

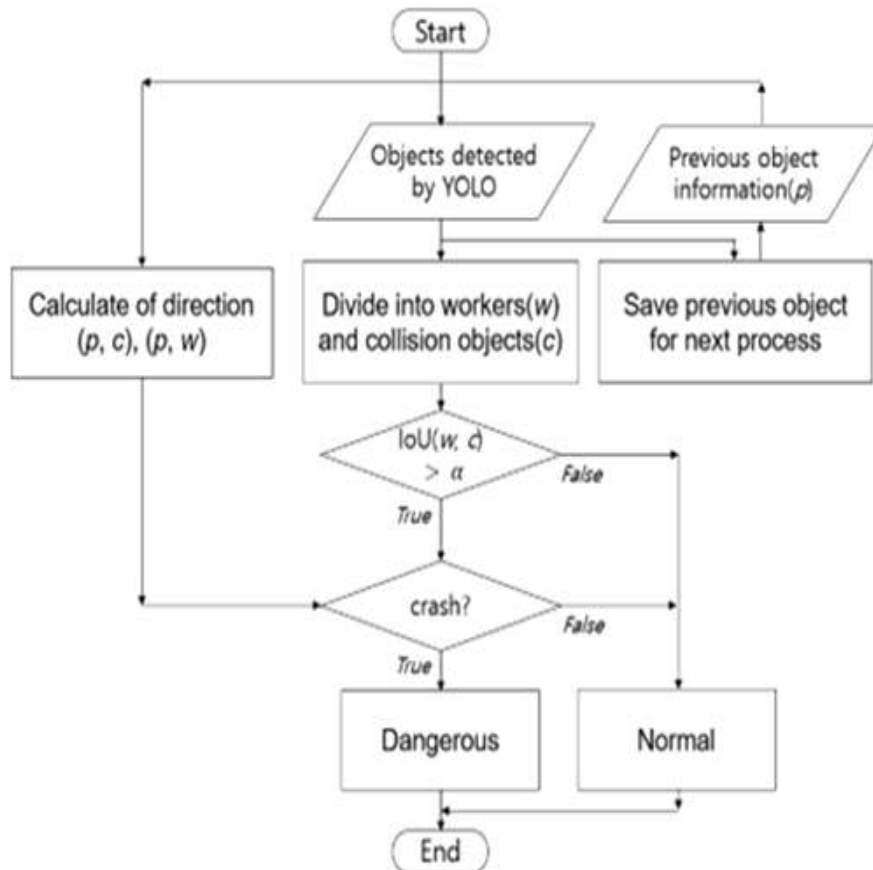


그림 22. Moving object collision risk determination algorithm

그림 22에서 먼저 YOLOv4로 탐지된 객체에 대해서 작업자와 이동 객체(트럭, 지게차 등)가 감지되면 이에 대한 좌푯값이 YOLOv4에 의해서 예측된 결과가 나온다. 그 두 좌푯값에 대한 겹침 정도, IoU(Intersection over Union) 값을 추출한다. 이 IoU 값이 0.15를 넘어가면 충돌한 경우로 본다. 아울러 충돌 이후 작업자의 급격한 가로와 세로 비율 변화를 반영하였다. 아래 그림 23은 충돌 직전과 충돌 이후에 검출된 결과를 표현한 것이다.



그림 23. Results detected before and after the crash

아래 그림 24와 25는 이동 객체가 충돌했는지를 판단하기 위해 객체의 방향과 서로 다가오는지 거리를 확인하는 코드를 표현한 것이다.

```
DIRECTION = ['LEFT', 'RIGHT', 'TOP', 'BOTTOM']
def detect_direction(prev_obj, next_obj):
    # prev_obj, next_obj info
    # (int(x), int(y), xmin, ymin, xmax, ymax)
    #   0       1       2       3       4       5

    # 중심점 x, y좌표
    #prev_dx, prev_dy = prev_obj[0] - prev_obj[0], prev_obj[1] - prev_obj[1]
    #next_dx, next_dy = next_obj[0] - next_obj[0], next_obj[1] - next_obj[1]

    ret = list()
    if float(prev_obj[2][0]) < float(next_obj[2][0]):
        ret.append(DIRECTION[1])
    else:
        ret.append(DIRECTION[0])

    if float(prev_obj[2][1]) < float(next_obj[2][1]):
        ret.append(DIRECTION[3])
    else:
        ret.append(DIRECTION[2])

    return ret
```

그림 24. Algorithm for determining the direction of movement of the previous frame object and the current frame object

```

def is_collision(worker, truck):
    # worker, truck example -> list('LEFT', 'TOP')
    if worker[0] == 'RIGHT' and truck[0] == 'LEFT':
        return True
    if worker[0] == 'LEFT' and truck[0] == 'RIGHT':
        return True

    if worker[1] == 'BOTTOM' and truck[1] == 'TOP':
        return True
    if worker[1] == 'TOP' and truck[1] == 'BOTTOM':
        return True

    return False

def is_disappear(screen_width, screen_height, obj):
    if 1 <= obj[2] <= screen_width - 1 or 1 <= obj[4] <= screen_height - 1:
        return False
    else:
        return True

```

그림 25. Collision detection algorithm and algorithm to determine if an object has disappeared

그림 24에서 이전 프레임 객체와 현재 프레임 객체의  $x$ 좌표를 비교해 왼쪽으로 이동했는지 오른쪽으로 이동했는지를 판단하며,  $y$ 좌표를 비교해 상단으로 이동했는지 하단으로 이동했는지를 판단한다. 그림 25에서는 판단한 결과를 작업자와 이동 객체가 정반대로 이동하고 있는 경우를 판단하는 함수이며, 그 이후에 객체가 전체 스크린의 가로와 세로를 침범하면 객체가 사라졌다고 판단하는 함수이다.

앞서 작업자와 이동 객체가 이전 프레임에서 다음 프레임으로 이동 시 같은 객체임을 인지해야 한다. YOLOv4에서는 단지 작업자, 이동 객체로만 인식할 뿐이다. 따라서, 이동 객체 유사도 검증 방법을 활용해야 한다. 이동 객체의 방향성과 유사도 산출을 위해 함수를 OpenCV를 활용해 별도로 독립적으로 구현하였다. 이에 관한 코드 사진은 그림 26, 27과 같다.

```

# 1. image cropping [y start : y end, x start : x end]
print(prev_obj[3], prev_obj[5], prev_obj[2], prev_obj[4])
print(next_obj[3], next_obj[5], next_obj[2], next_obj[4])

prev_obj_y1 = 0
if prev_obj[3] < 0: prev_obj_y1 = 0
else: prev_obj_y1 = prev_obj[3]

prev_obj_x1 = 0
if prev_obj[2] < 0: prev_obj_x1 = 0
else: prev_obj_x1 = prev_obj[2]

next_obj_y1 = 0
if next_obj[3] < 0: next_obj_y1 = 0
else: next_obj_y1 = next_obj[3]

next_obj_x1 = 0
if next_obj[2] < 0: next_obj_x1 = 0
else: next_obj_x1 = next_obj[2]

prev_obj_img = prev_img[prev_obj_y1:prev_obj[5], prev_obj_x1:prev_obj[4]].copy()
next_obj_img = next_img[next_obj_y1:next_obj[5], next_obj_x1:next_obj[4]].copy()

```

그림 26. Object preprocessing code for calculating similarity

```

# 2. image simliarity anlaysis
hsv_prev_img = cv2.cvtColor(prev_obj_img, cv2.COLOR_BGR2HSV)
hist_prev_img = cv2.calcHist([hsv_prev_img], [0, 1], None, [180, 256], [0, 180, 0, 256])
cv2.normalize(hist_prev_img, hist_prev_img, 0, 1, cv2.NORM_MINMAX)

hsv_next_img = cv2.cvtColor(next_obj_img, cv2.COLOR_BGR2HSV)
hist_next_img = cv2.calcHist([hsv_next_img], [0, 1], None, [180, 256], [0, 180, 0, 256])
cv2.normalize(hist_next_img, hist_next_img, 0, 1, cv2.NORM_MINMAX)

simliar = cv2.compareHist(hist_next_img, hist_prev_img, cv2.HISTCMP_INTERSECT)
simliar = simliar / np.sum(hist_next_img)
print(f'next prev simliarity = {simliar}')

if simliar >= SIMLIARITY_THRESHOLD:
    print('next prev same!!')
    return True
else:
    return False

```

그림 27. Similarity verification code with OpenCV

그림 26을 보면 YOLOv4에서 나온 결과는 작업자, 이동 객체가 여러 개가 나타난다. 이 경우 Enumerate한 객체 식별자가 전달된다. 식별자가

프레임마다 계속 바뀌기 때문에 이전 프레임과 현재 프레임이 등장하는 작업자, 이동 객체가 동일한 객체인지 계속 OpenCV를 활용한다.

여기서 OpenCV를 활용하는 것은 히스토그램 유사도(Histogram similarity)를 활용한다[37]. OpenCV에서 제공하는 히스토그램 유사도 기능은 그림 28과 같다. 본 연구에서는 바타차야 거리 유사도(Bhattacharyya distance)를 활용하였다. OpenCV의 히스토그램 유사도 검증은 RGB(Red, Green, Blue) 색 공간을 활용하지 않고 HSV(Hue, Saturation, Value) 색 공간을 활용한다. 바타차야 거리 유사도 수식은 그림 28, 4번째 행에 있다.

| Enumerator                                                |                                                                                                                                                                                                                                                   |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HISTCMP_CORREL<br>Python: cv.HISTCMP_CORREL               | Correlation $d(H_1, H_2) = \frac{\sum_I (H_1(I) - \bar{H}_1)(H_2(I) - \bar{H}_2)}{\sqrt{\sum_I (H_1(I) - \bar{H}_1)^2 \sum_I (H_2(I) - \bar{H}_2)^2}}$ where $\bar{H}_k = \frac{1}{N} \sum_I H_k(I)$ and $N$ is a total number of histogram bins. |
| HISTCMP_CHISQR<br>Python: cv.HISTCMP_CHISQR               | Chi-Square $d(H_1, H_2) = \sum_I \frac{(H_1(I) - H_2(I))^2}{H_1(I)}$                                                                                                                                                                              |
| HISTCMP_INTERSECT<br>Python: cv.HISTCMP_INTERSECT         | Intersection $d(H_1, H_2) = \sum_I \min(H_1(I), H_2(I))$                                                                                                                                                                                          |
| HISTCMP_BHATTACHARYYA<br>Python: cv.HISTCMP_BHATTACHARYYA | Bhattacharyya distance (In fact, OpenCV computes Hellinger distance, which is related to Bhattacharyya coefficient) $d(H_1, H_2) = \sqrt{1 - \frac{1}{\sqrt{H_1 H_2} N^2} \sum_I \sqrt{H_1(I) \cdot H_2(I)}}$                                     |
| HISTCMP_HELLINGER<br>Python: cv.HISTCMP_HELLINGER         | Synonym for HISTCMP_BHATTACHARYYA.                                                                                                                                                                                                                |
| HISTCMP_CHISQR_ALT<br>Python: cv.HISTCMP_CHISQR_ALT       | Alternative Chi-Square $d(H_1, H_2) = 2 * \sum_I \frac{(H_1(I) - H_2(I))^2}{H_1(I) + H_2(I)}$ This alternative formula is regularly used for texture comparison. See e.g. [178]                                                                   |
| HISTCMP_KL_DIV<br>Python: cv.HISTCMP_KL_DIV               | Kullback-Leibler divergence $d(H_1, H_2) = \sum_I H_1(I) \log \left( \frac{H_1(I)}{H_2(I)} \right)$                                                                                                                                               |

그림 28. Histogram similarity features provided by OpenCV

### 4.3 위험판정 성능 측정과 결과

검출된 객체 기반 상황 판정 성능 측정을 위해 K-fold 교차 방법을 활용하였다. 이에 관한 흐름도는 그림 29와 같다.

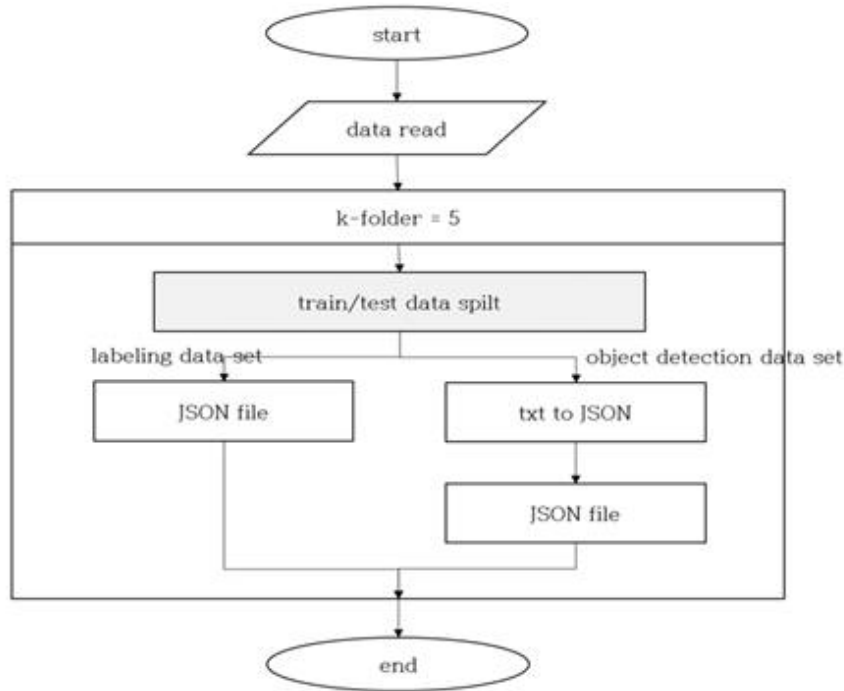


그림 29. Performance measurement flowchart using the k-fold cross validation method

그림 29에서 첫 번째로 객체 검출 데이터셋을 확인하고 시험 데이터의 위험 상황별 객체 검출 결과가 담긴 텍스트 파일과 시험 데이터의 위험 상황 판정 결과가 담긴 JSON 파일 결과를 확인한다. 두 번째로 시험 케이스를 k-fold 교차 방법을 활용해 위험 상황별 데이터로 분리한다. 세 번째는 시험 케이스 별로 모델에서 출력된 상황 판정 결과를 생성한다. 네 번째로 위험 상황별 상황 판정 결과와 객체 검출, 위험 상황 판정 결과가 담긴 텍스트 파일과 JSON 파일을 활용해 상황 판정 결과를 비교한다. 마지막으로 각 K-case별 위험 상황별 RMSE(Root Mean Squared Error)를 종합해 평균 성능을 확인한다. 이에 대한 판정 결과를 종합하면 표 13과 같다. 이에 대한 그래프는 그림 30과 같다.

표 13. Object detection-based contextual judgment performance results

| No.          | Number of frame | JSON record | Number of dangerous situations | Percentage of dangerous situation | Number of detection | Accuracy (%) |
|--------------|-----------------|-------------|--------------------------------|-----------------------------------|---------------------|--------------|
| 1            | 17,792          | 17,792      | 172                            | 1                                 | 172                 | 100          |
| 2            | 15,000          | 15,000      | 5,381                          | 36                                | 5,381               | 100          |
| 4            | 17,792          | 17,792      | 17,792                         | 100                               | 17,792              | 100          |
| 5            | 9,072           | 9,072       | 6,697                          | 74                                | 6,697               | 100          |
| 6            | 15,000          | 15,000      | 6,116                          | 41                                | 6,116               | 100          |
| 7            | 9,141           | 9,141       | 6,627                          | 72                                | 6,627               | 100          |
| 8            | 10,459          | 10,459      | 7,096                          | 68                                | 7,096               | 100          |
| <b>Total</b> | 94,256          | 94,256      | 49,881                         | 53                                | 49,881              | 100          |

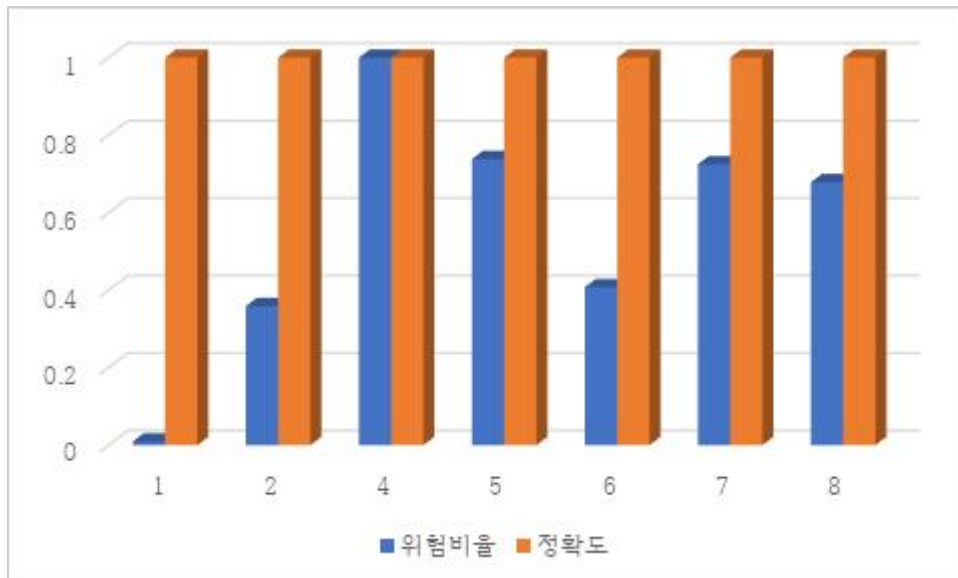


그림 30. Object detection-based contextual judgment performance graph

추가로 반복적 시험을 통해 객체 검출 지연시간을 산출하였다. 초당 프레임은 25로 진행하였다. 총횟수는 13번 진행했으며 각 위험 상황 시나리오 8가지에 대해 실험하였다. 평균값, 최솟값, 최댓값을 산출하였다. 이에 관한 결과는 표 14와 같으며, 이에 관한 그래프는 그림 31과 같다.

표 14. Object detection latency calculation results

| No. | Dangerous situation No. |          |          |          |          |          |          |
|-----|-------------------------|----------|----------|----------|----------|----------|----------|
|     | 1                       | 2        | 4        | 5        | 6        | 7        | 8        |
| 1   | 1.91E-05                | 2.49E-05 | 1.91E-05 | 6.31E-05 | 9.21E-06 | 3.72E-05 | 4.25E-05 |
| 2   | 1.14E-05                | 1.94E-05 | 1.14E-05 | 2.39E-05 | 1.22E-05 | 2.20E-05 | 3.24E-05 |
| 3   | 7.48E-06                | 1.15E-05 | 7.48E-06 | 1.67E-05 | 8.28E-06 | 1.53E-05 | 1.36E-05 |
| 4   | 5.75E-06                | 1.45E-05 | 5.75E-06 | 1.26E-05 | 8.62E-06 | 1.22E-05 | 1.27E-05 |
| 5   | 4.99E-06                | 6.52E-06 | 4.99E-06 | 1.03E-05 | 7.99E-06 | 1.29E-05 | 1.10E-05 |
| 6   | 3.83E-06                | 3.95E-06 | 3.83E-06 | 8.45E-06 | 9.71E-06 | 6.74E-06 | 7.05E-06 |
| 7   | 3.23E-06                | 3.56E-06 | 3.23E-06 | 7.24E-06 | 7.08E-06 | 6.74E-06 | 6.21E-06 |
| 8   | 2.83E-06                | 4.44E-06 | 2.83E-06 | 6.11E-06 | 8.77E-06 | 6.80E-06 | 5.50E-06 |
| 9   | 2.52E-06                | 2.61E-06 | 2.52E-06 | 5.66E-06 | 6.68E-06 | 5.20E-06 | 5.00E-06 |
| 10  | 1.95E-06                | 2.43E-06 | 1.95E-06 | 4.90E-06 | 6.38E-06 | 4.98E-06 | 6.17E-06 |
| 11  | 2.07E-06                | 2.42E-06 | 2.07E-06 | 3.93E-06 | 6.25E-06 | 3.70E-06 | 4.41E-06 |
| 12  | 1.90E-06                | 5.79E-06 | 1.90E-06 | 4.29E-06 | 6.17E-06 | 3.97E-06 | 3.81E-06 |
| 13  | 1.73E-06                | 5.03E-06 | 1.73E-06 | 3.96E-06 | 5.85E-06 | 3.78E-06 | 3.40E-06 |
| Avg | 5.29E-06                | 8.23E-06 | 5.29E-06 | 1.32E-05 | 7.94E-06 | 1.09E-05 | 1.18E-05 |
| Min | 1.73E-06                | 2.42E-06 | 1.73E-06 | 3.93E-06 | 5.85E-06 | 3.70E-06 | 3.40E-06 |
| Max | 1.91E-05                | 2.49E-05 | 1.91E-05 | 6.31E-05 | 1.22E-05 | 3.72E-05 | 4.25E-05 |

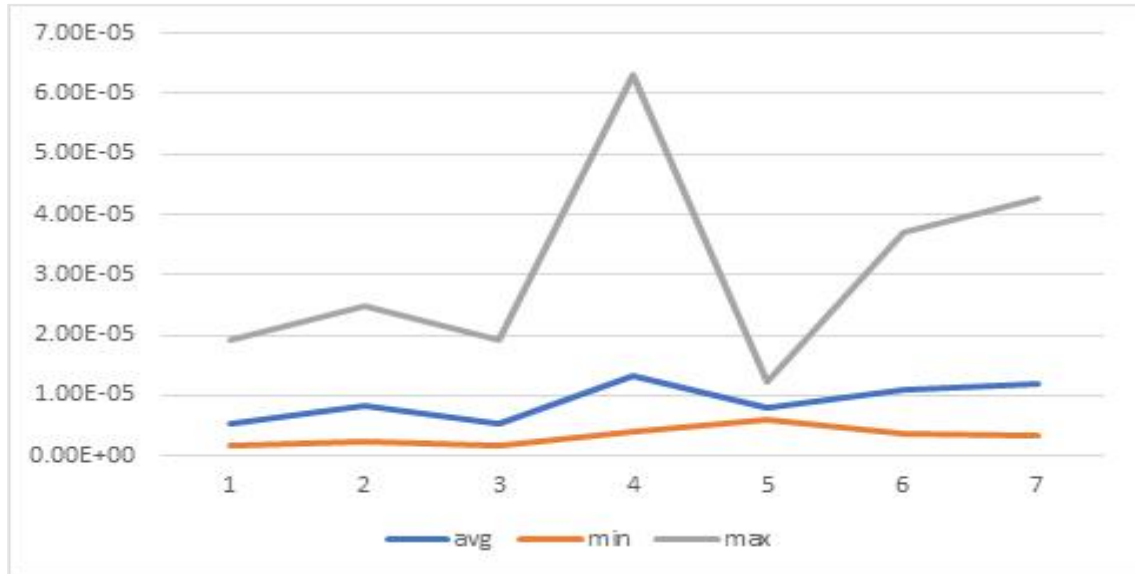


그림 31. Object detection latency calculation result graph

표 14와 그림 31을 통해 영상 검출 지연시간을 프레임 단위로 측정한 결과  $1.73\text{E}-06$ 초에서  $6.31\text{E}-05$ 초 사이에 산출되었다. 초당 프레임을 25로 했을 때도 영상처리가 불가능한 수준은 아니었다. 여기서 영상처리가 불가능한 수준은 처리시간이 초당 프레임 입력시간보다 큰 경우를 의미한다.

이번에는 위험 상황별 객체 검출이 얼마나 잘되는지를 판단하기 위해 검출 IoU 값을 조정하여 성능을 판단하였다. 아래 그림 32는 mAP@0.5 일 때 최고의 성능을 보였을 때(Best), 평균 손실 값(Avg loss), 이미지 처리 비율/Images)을 그래프로 표현한 것이다.

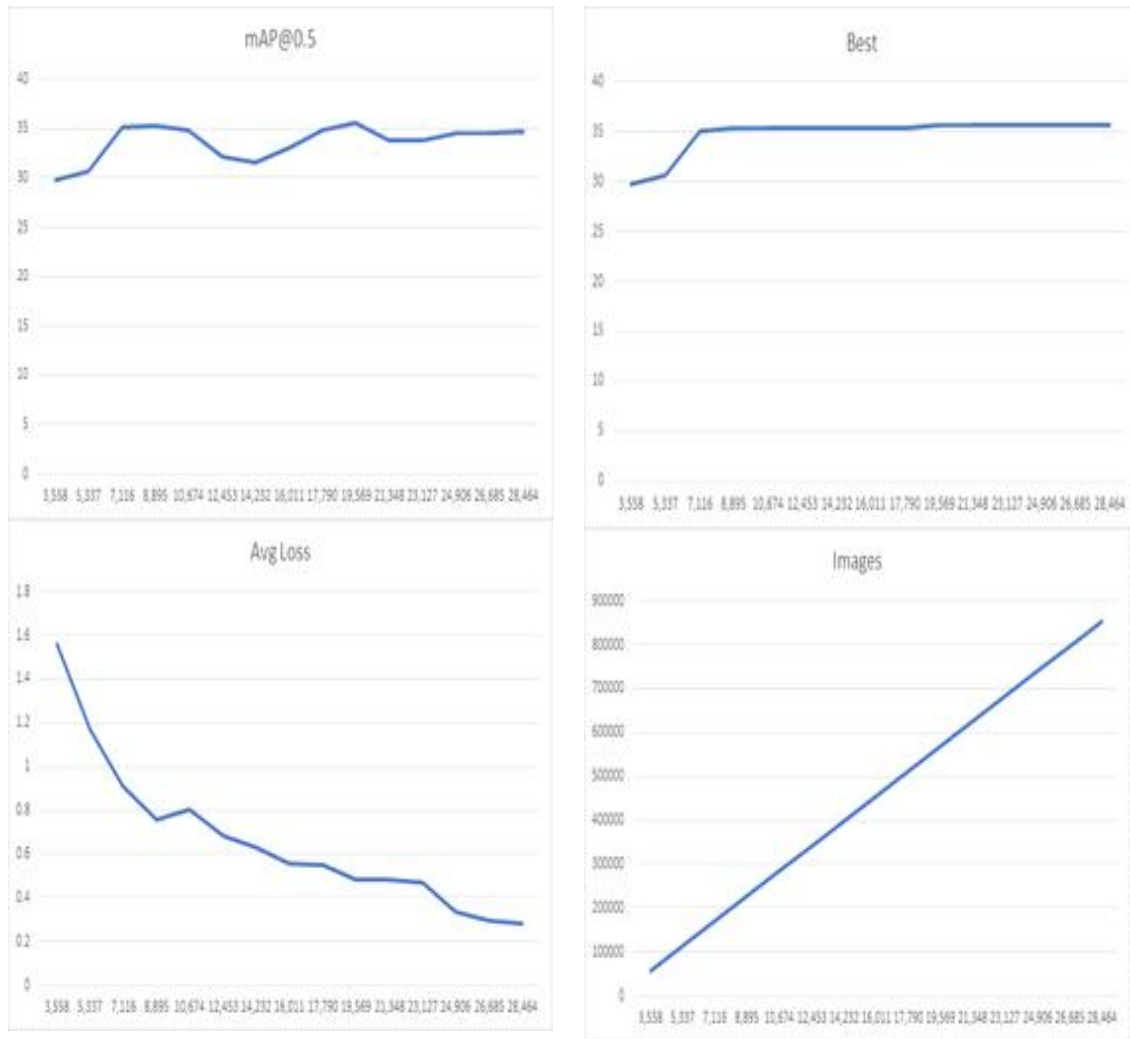


그림 32. Adjust the detection IoU value to determine performance

아래 표 15는 객체 검출 정확도를 산출한 결과를 의미한다. IoU가 0.25일 때 99.08의 성능으로 가장 높은 성능을 보였다.

표 15. The result of calculating the object detection accuracy

| No. | Object name      | mAP@0.25 |       |       |       |       |       |       |       |
|-----|------------------|----------|-------|-------|-------|-------|-------|-------|-------|
|     |                  | 1        | 2     | 4     | 5     | 6     | 7     | 8     | mAP   |
| 1   | Woker            | 98.88    | 99.23 | 98.88 | 100   | 65.61 | -     | -     | 94.65 |
| 2   | Object           | 99.95    | 99.99 | 99.95 | -     | -     | -     | -     | 99.96 |
| 3   | Cart             | 100      |       | 100   | -     | -     | -     | -     | 100   |
| 4   | Fork_Lift        | 95.33    |       | 95.33 | -     | 98.24 | -     | -     | 96.3  |
| 5   | Truck            | -        | -     | -     | -     | 100   | -     | -     | 100   |
| 6   | Sanitary_Cap     | -        | -     | -     | 100   | -     | -     | 100   | 100   |
| 7   | Sanitary_Clothes | -        | -     | -     | 100   | -     | 100   | -     | 100   |
| 8   | Mask             | -        | -     | -     | -     | -     | 100   | 100   | 100   |
| 9   | Gloves           | -        | -     | -     | 99.91 | -     | 99.96 | 99.96 | 99.94 |
| 10  | Danger Zone      | 100      | 100   | 100   | -     | -     | -     | -     | 100   |

위 표 15에서 1부터 8까지 시나리오를 의미한다. mAP가 0.25 이상일 때 객체로 본다는 의미이다. 위험 구역(danger zone)과 같은 구역검출, 위생 장구와 안전 장구와 같은 위생모자, 위생복 등의 경우 100%의 검출 성능을 보였다. 실외 환경 작업자 검출 과정인 6번 시나리오에서는 최저 검출률 65.61이 산출되었다. 실증 사업장의 경우 IoU 설정값이 0.25인 값에서 더 높였을 때 검출 정확도가 낮게 산출되는 문제가 발생하기 때문에 IoU가 낮은 수준에서 객체를 검출하고 위험 상황을 판정할 수 있도록 하였다.

#### 4.4 관제시스템 설계

본 논문에서 제안하는 시스템은 산업현장에서 발생할 수 있는 안전사고를 방지하기 위한 것이다. 산업현장에서 설치된 CCTV로부터 수집된 영상데이터와 딥러닝 모델을 통합해 작업 중 위험 여부를 관리자에게 알려줄 수 있고, 현장 작업자에게 경보 알람을 줄 수도 있다. 산업안전사고 방지 시스템에 대한 전체적인 통합적 설계 구조도는 그림 33과 같다.



그림 33. A integrated design flowchart for an industrial accident prevention system

그림 33에서 활용되는 시스템의 각각의 기능에 대한 정의는 다음과 같다. 먼저 데이터 수집 기능은 실증 사업장에서 CCTV로 촬영한 영상데이터를 수집하는 기능이 필요하다. 수집된 데이터에 대해 픽셀 크기, 이미지 보정 등의 전처리 기능이 필요하며, 이를 통해 작업 현장 내에서 객체와 구역을 검출하는 기능이 필요하다. 이 중에서 특히 이동 객체(작업자, 트럭, 카트 등)의 속도나 방향을 계산하는 이동 객체의 상태를 검출하는 기능이 있어야 하며, 거리 예측 기능도 필요하다.

이를 종합적으로 판단해 시나리오별 위험 상황을 인식하는 위험판정 기능이 있어야 한다. 아울러 실시간 영상 모니터링과 위험 상황 판정을 통합적으로 관제할 수 있는 관제시스템이 필요하다. 이후 위험 경보장치나 관제시스템을 관리하는 담당자, 안전관리 담당자에게 통보할 수 있는 기능 또한 제공해야 한다.

아래 표 16과 17은 각각 관제시스템이 설치된 환경과 활용된 소프트웨어와 라이브러리를 표현한 것이다. 표 18은 IP 카메라의 제원을 표현한 것이다.

표 16. Environment where the control system is installed

|                            |                  |                                           |
|----------------------------|------------------|-------------------------------------------|
| <b>Hardware<br/>Server</b> | <b>CPU</b>       | Intel core i9-12gen (3.4Ghz)              |
|                            | <b>RAM</b>       | 32GB                                      |
|                            | <b>SSD</b>       | 1TB                                       |
|                            | <b>Graphic</b>   | Nvidia RTX3090 (VRAM: 24GB)               |
| <b>Network</b>             | <b>Switch</b>    | 1Gbps (PoE supported)                     |
| <b>Etc.</b>                | <b>IP camera</b> | HIKVISION Network camera (DS-2CD3045G0-I) |

표 17. Software and libraries utilized in the control system

| Software and library name | Version | Content              | Note |
|---------------------------|---------|----------------------|------|
| Ubuntu Desktop            | 20.04   | Operating system     |      |
| Python3                   | 3.8     | Development language |      |
| tensorflow                | 2.7     |                      |      |
| CUDA                      | 10.1    |                      |      |
| CUDA Toolkit              | 10.1    |                      |      |
| YoloV4                    | v4      |                      |      |
| OpenCV                    | 4.2     |                      |      |
| screeninfo                | 0.9.2   | Python library       | Lib  |
| numpy                     | 1.22.2  | Python library       | Lib  |
| scipy                     | 1.6.1   | Python library       | Lib  |
| absl-py                   | 0.11.0  | Python library       | Lib  |
| keras-Preprocessing       | 1.1.2   | Python library       | Lib  |
| Mako                      | 1.0.7   | Python library       | Lib  |
| pyparsing                 | 3.0.7   | Python library       | Lib  |
| python-dateutil           | 2.8.2   | Python library       | Lib  |
| requests                  | 2.25.1  | Python library       | Lib  |
| screen-resolution-extra   | 0.1     | Python library       | Lib  |
| simplejson                | 3.13.2  | Python library       | Lib  |
| system-service            | 0.3     | Python library       | Lib  |
| tensorboard               | 2.4.1   | Python library       | Lib  |
| systemd-python            | 234     | Python library       | Lib  |
| tensorflow-estimator      | 2.4.0   | Python library       | Lib  |
| matplotlib                | 2.5.1   | Python library       | Lib  |

## X 18. IP camera specifications

| Camera features          | Content                                                                                                                                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Image Sensor             | 1/2" Progressive Scan CMOS                                                                                                                                                    |
| Min. Illumination        | Color: 0.008 Lux @ (F1.2, AGC ON), 0.014 Lux @ (F1.6, AGC ON)                                                                                                                 |
| Shutter Speed            | 1/3 s to 1/100,000 s                                                                                                                                                          |
| Wide Dynamic Range       | 120dB                                                                                                                                                                         |
| Day & Night              | IR cut filter                                                                                                                                                                 |
| Angle Adjustment         | Pan: 0° to 360°, tilt: 0° to 90°, rotate: 0° to 360°                                                                                                                          |
| Max. Resolution          | 3840 × 2160                                                                                                                                                                   |
| Main Stream              | 50Hz: 20 fps (3840 × 2160), 25 fps (3072 × 1728, 2560 × 1440, 1920 × 1080, 1280 × 720),60Hz: 20 fps (3840 × 2160), 30 fps (3072 × 1728, 2560 × 1440, 1920 × 1080, 1280 × 720) |
| Sub Stream               | 50Hz: 25fps (640 × 480, 640 × 360, 320 × 240),60Hz: 30fps (640 × 480, 640 × 360, 320 × 240)                                                                                   |
| Third Stream             | 50Hz: 25fps (1280 × 720, 640 × 360, 352 × 288),60Hz: 30fps (1280 × 720, 640 × 360, 352 × 240)                                                                                 |
| Video Compression        | Main stream: H.265/H.264,Sub-stream: H.265/H.264/MJPEG,Third stream: H.265/H.264                                                                                              |
| Video Bit Rate           | 32 Kbps to 16 Mbps                                                                                                                                                            |
| H.264 Type               | Main Profile/High Profile                                                                                                                                                     |
| H.265 Type               | Main Profile                                                                                                                                                                  |
| H.264+                   | Main stream supports                                                                                                                                                          |
| H.265+                   | Main stream supports                                                                                                                                                          |
| Region Of Interest (ROI) | Support 1 fixed region for main stream and sub-stream separately                                                                                                              |
| Simultaneous Live View   | Up to 6 channels                                                                                                                                                              |
| API                      | ONVIF (PROFILE S, PROFILE G, PROFILE T), ISAPI, SDK, Ehome                                                                                                                    |
| Protocols                | TCP/IP, UDP, ICMP, HTTP, HTTPS, FTP, DHCP, DNS, DDNS, RTP, RTSP, RTCP, PPPoE, NTP, UPnP, SMTP, SNMP, IGMP, 802.1X, QoS, IPv6, UDP, Bonjour, SSL/TLS, WebSocket, WebSockets    |
| User/Host                | Up to 32 users,3 levels: Administrator, Operator and User                                                                                                                     |

| Camera features         | Content                                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Image Enhancement       | BLC/3D DNR/HLC                                                                                                                    |
| Image Settings          | Rotate mode, saturation, brightness, contrast, sharpness, AGC, and white balance are adjustable by client software or web browser |
| On-Board Storage        | Built-in micro SD/SDHC/SDXC slot, up to 128 GB                                                                                    |
| Communication Interface | 1 RJ45 10M/100M self-adaptive Ethernet port                                                                                       |

IP 카메라는 위 표 18에서 언급한 카메라가 어떤 제조사의 제품을 사용해도 문제가 없지만 IP 카메라의 제원에 따라 영상의 품질이 저하되거나 너무 고해상도 영상을 수신할 때 서버의 영상처리 속도에 따라 영상 지연시간이 발생할 수 있다. IP 카메라와 Python, OpenCV 영상처리 라이브러리와 통신하기 위해 RTSP(Real-Time Streaming Protocol) 형식의 도메인을 지원해야 한다. 아래는 RTSP 도메인의 예를 보인 것이다.

rtsp://user\_id:password@IP\_address:port/Streaming/channels/101

관제시스템은 IP 카메라에서 영상을 수신하기 위해 네트워크를 구성해야 한다. 카메라에 전원을 공급하기 위해서는 PoE(Power of Ethernet) 스위치를 사용하며 서버, 경보장치는 별도의 전원이 필요하므로 PoE 스위치에 연결하지 않고 일반 스위치에 연결한다.

IP 카메라의 개수는 서버의 영상처리 성능을 고려해 설치해야 하며 표 16에서 언급한 사양의 서버에서 4대의 카메라 영상을 수신해 영상에 등장하는 객체를 검출할 수 있다. 카메라를 추가하면 서버에서 카메라 정보를 추가하여 바로 화면에 출력할 수 있다. 경보장치의 경우 3개의 램프와 오디오 신호가 출력할 수 있는 램프이며 일반적으로는 녹색등이 점등되고 위험 상황이 발생하면 붉은색 램프가 점등되며 경보음도 울리게 된다. 경보장치도 서버와 네트워크 통신을 통해 정보를 주고받는다. 아래 그림 34는 관제시스템의 구성도를 표현한 것이다.

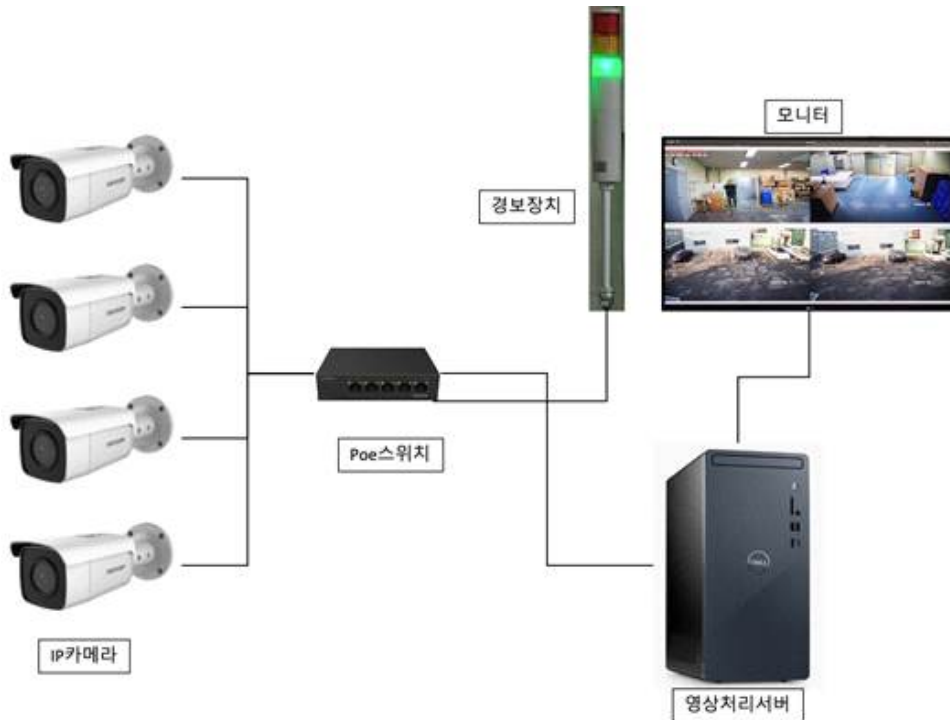


그림 34. Control system hardware and network diagram

관제시스템의 주요 기능은 실시간 모니터링, 과거 영상조회, 위험 상황 발생 이력 관리, 객체 검출, 오디오 신호 생성, 사업자와 사용자 관리를 위한 주요 기능을 제공한다. 먼저 실시간 모니터링은 관제시스템의 대시보드로 위험 상황을 실시간으로 영상으로 출력하고 객체를 검출한 결과 또한 출력한다. 위험 상황이 발생하면 화면에 위험 상황을 붉은색으로 표시하고 경보장치에 알람을 전송한다.

과거 영상조회 기능은 실시간 모니터링 화면을 일자별, 시간별로 저장소에 저장하여 일자별, 시간별로 과거 영상을 조회할 수 있도록 한다. 위험 상황 발생 이력 관리는 실시간 모니터링에서 검출한 위험 상황을 위험 상황의 종류, 시간 등 이력을 저장소에 저장하여 향후 이 내용을 바탕으로 과거 영상을 조회할 수 있다.

객체 검출 기능은 관제시스템이 CCTV의 영상을 처리하는 기능만을 수행하는 것이 아니라 객체의 검출과 위험 상황 판단을 인공지능에 기반한 객체 검출 알고리즘, 모델을 활용한 것이다. 앞서 4.1절부터 4.3절까지의 내용이며 본 관제시스템의 핵심 기능이라고 할 수 있다. 객체 검출 기능의 경우 모듈로 개발하였기 때문에 학습데이터의 변경이나 상황 판단 알고리즘을 변경하면 바로 적용할 수 있도록 개발하였다.

오디오 신호 생성 기능은 실시간 모니터링 기능에서 위험 상황이 발생하면 경보/알람 장치를 통해 붉은색 등이 점등되어 위험 상황을 알리고 알람 신호가 발생하게 된다. 위험 상황이 종료되면 알람 신호가 종료되고 초록색 등이 점등된다.

사업자와 사용자 관리 기능은 본 관제시스템이 CCTV 카메라를 서버가 처리할 수 있는 만큼 개수를 증설할 수 있으므로 카메라가 추가하면 설정 화면에서 정보를 등록해 화면을 추가할 수 있으며, 사용자는 관리자와 일반사용자를 등록해 사용할 수 있다.

아래 그림 35에서 37까지는 관제시스템의 모듈들이 동작하는 순서도를 3분할로 표현한 것이다.

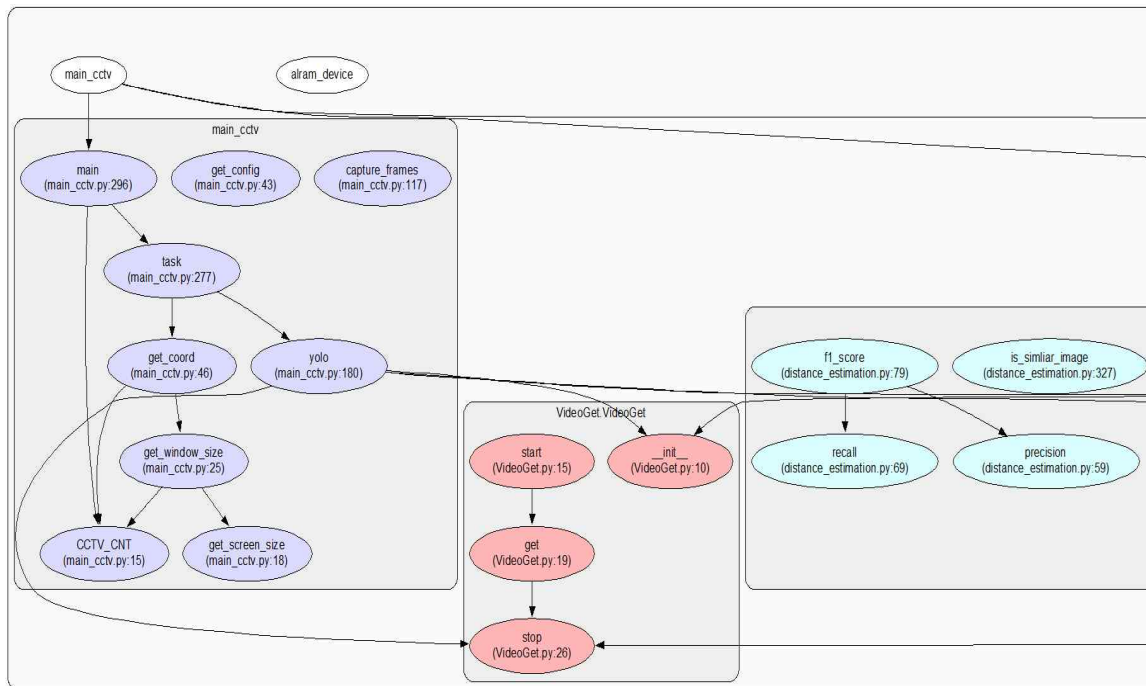


그림 35. Flowchart of how the modules in the control system work 1

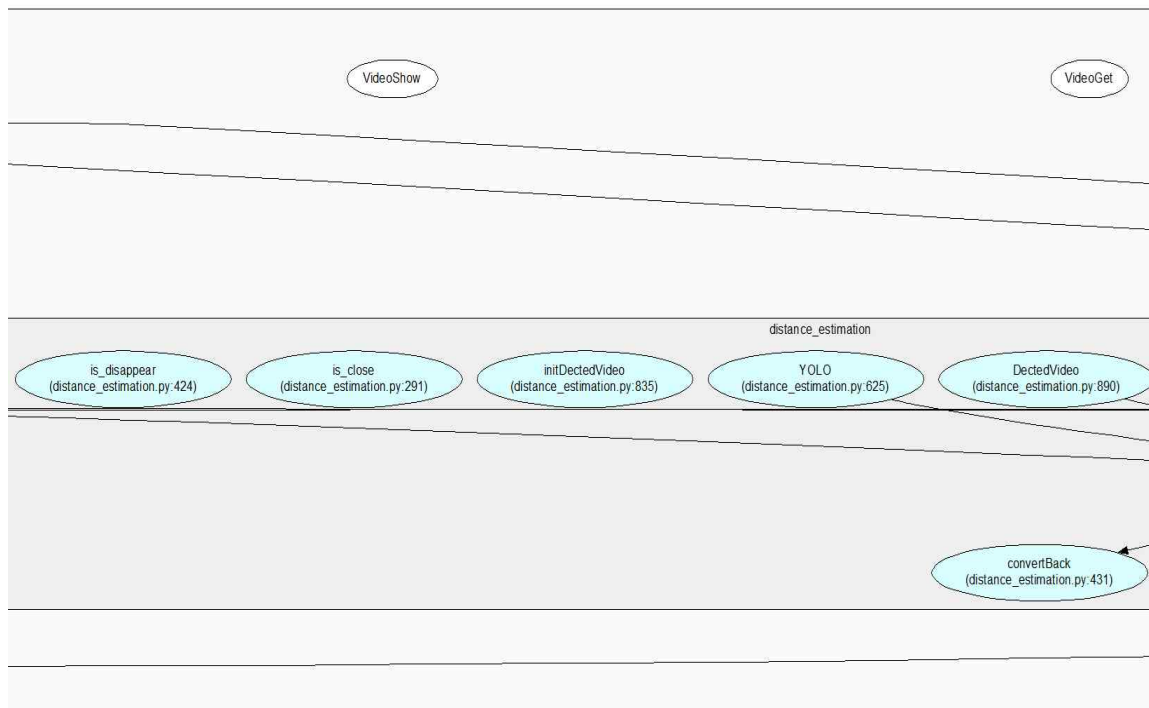


그림 36. Flowchart of how the modules in the control system work 2

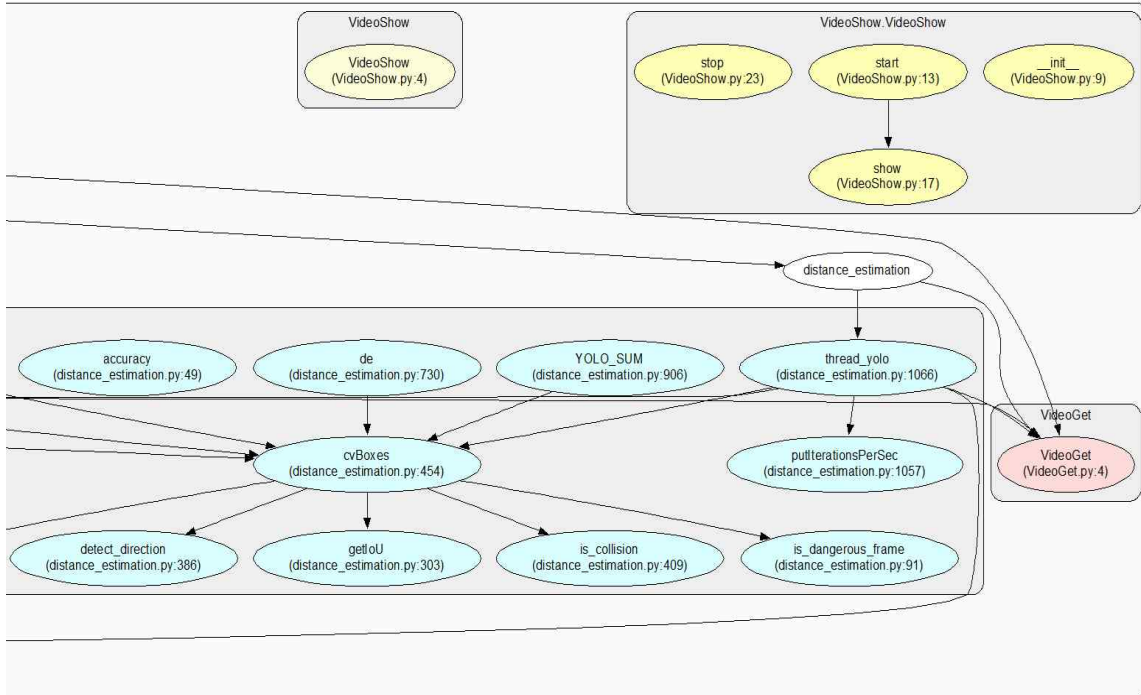


그림 37. Flowchart of how the modules in the control system work 3

먼저 그림 35에서 main\_cctv 모듈이 실행되면 task를 생성하여 get\_coord와 yolo가 동시에 호출된다. get\_coord 모듈은 객체의 현재 좌표를 yolo 모듈에서 가져오며 get\_window\_size 모듈을 호출한다. 이 모듈은 yolo 모듈에서 나온 객체의 좌표를 통해 객체의 가로, 세로 크기와 왼쪽상단 좌표로 변경해주는 역할을 한다.

이를 통해 VideoGet 모듈에서 CCTV에서 추출한 영상을 프레임 단위로 멀티스레드(Multi-thread) 방식으로 추출하여 이를 사용자 화면에 출력하게 한다. 이 모듈에 대한 설명은 후술할 그림 40에서 더 설명하겠다.

이후 객체가 충돌했는지, 위험 상황 시나리오를 위반 여부를 검사하기 위해 distance\_estimation 모듈이 호출된다. 이 모듈은 그림 35의 오른쪽 부분에서 그림 36, 그림 37의 하단 부분으로 하늘색으로 표시되어 있다. 이 모듈에는 객체가 등장했다가 사라졌는지를 확인하는 is\_disappear 모듈, 작업자 객체와 트럭과 같은 객체의 거리가 가까워 충돌했는지 판단

하는 is\_collision 모듈이 있으며 거리로 판단하는 is\_close 모듈과 IoU를 바탕으로 충돌했는지를 판단하는 getIoU 모듈이 있다. 이러한 모듈들은 앞서 그림 24부터 27에서 언급하였다. 이 모듈의 결과가 다시 VideoGet 모듈을 통해 결과로 출력되게 된다. 이에 관한 순서도를 더욱 간단히 표현하면 그림 38에서 39와 같다.

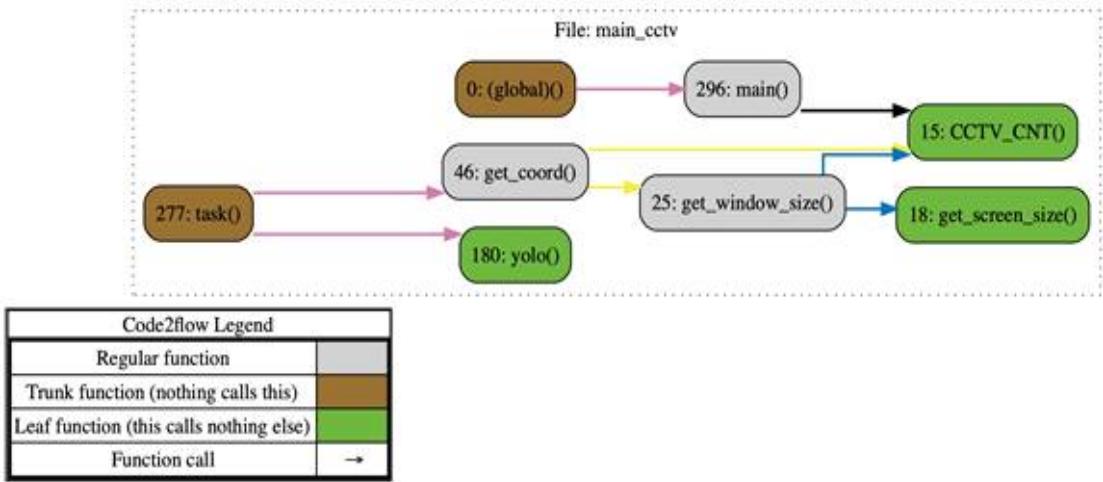


그림 38. Flow diagram of the control system main CCTV

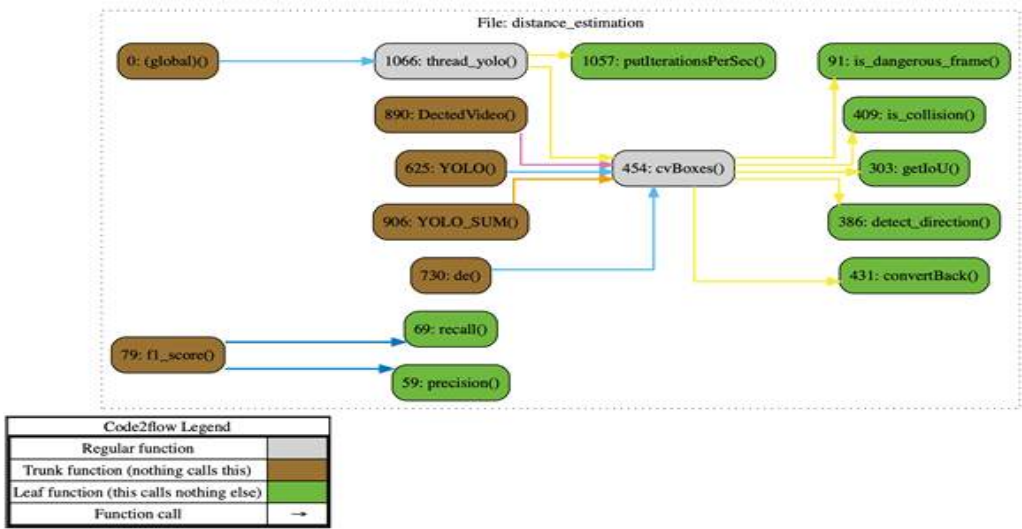


그림 39. Flow diagram of control system object detection and hazardous situation judgment

그림 38은 관제시스템의 메인 CCTV 모듈을 순서도로 표현한 것이다. 메인 CCTV 모듈은 설정 파일을 읽어서 화면을 분할하고 처리하여야 할 영상의 프로세스를 생성해 화면으로 출력한다.

그림 39는 관제시스템이 객체 검출과 위험 상황 판단을 위한 순서도를 표현한 것으로 위험 상황 판단 및 객체 검출 모듈이 메인에서 생성한 프로세스에 의해 실행되는 모듈이다. 하나의 영상에서 하나의 객체 검출 모듈이 프로세스 형태로 생성되어 처리되며 여러 개의 영상을 하나의 프로세스로 처리하면 많은 성능 저하가 발생한다. 아래 그림 40은 VideoGet 모듈에 관한 순서도이다.

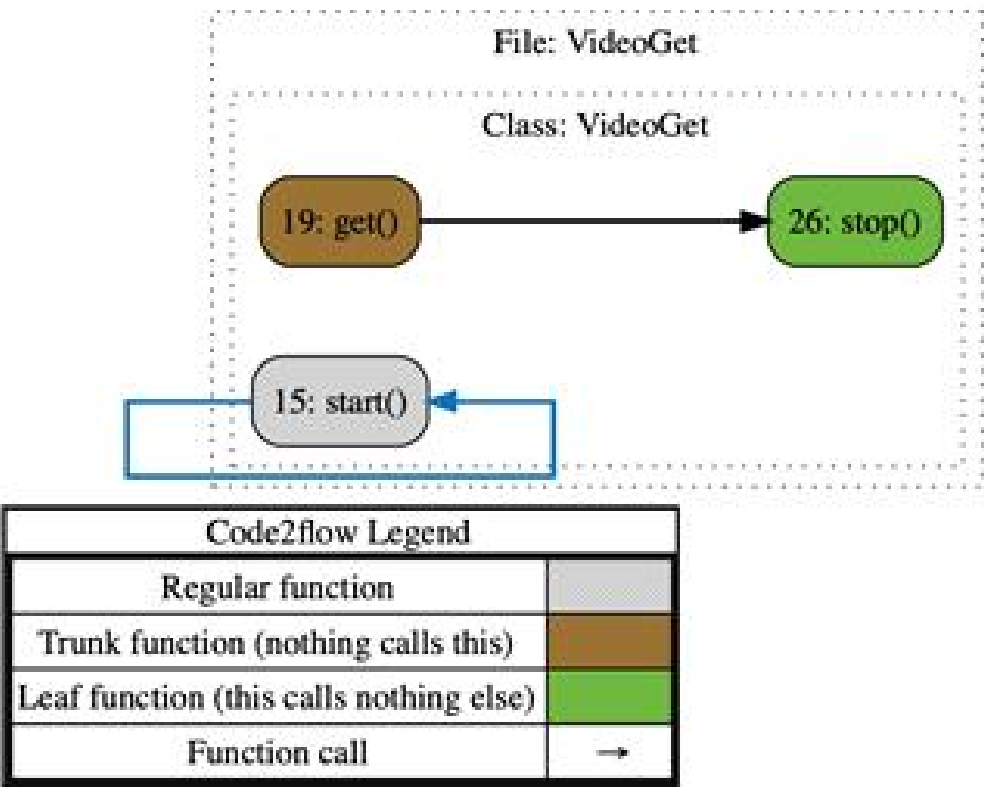


그림 40. Flowchart for the VideoGet module

그림 40에서 VideoGet 모듈은 main 모듈에서 VideoGet 모듈을 프로세스로 생성한다. CCTV로부터 영상을 수신하고 객체 검출 프로세스에서 영상을 처리할 수 있도록 변환하여 객체 검출 모듈로 영상을 전송한다. CCTV 영상을 하나의 프로세스로 처리하면 CCTV 영상 출력 시간에 많은 지연시간이 발생하기 때문에, CCTV 영상 하나에 대해 처리하는 프로세스를 생성해 성능을 극대화해야 한다. 이러한 문제점을 해결하기 위해 Python 코드 실행시간 최적화 기법, 리스트 열거(list comprehension), 문자열 병합 속도 최적화, 멀티프로세싱(Multi-processing) 기법을 적용하였다.

먼저 Python 코드 실행시간 최적화 기법은 Python 자체가 인터프리터 언어이기 때문에 제약조건이 많다. 인터프리터 언어는 코드를 한 번에 한 줄씩 읽어 실행하므로 컴파일러 언어에 비해 실행 속도가 느리다는 단점이 존재한다. 미리 소스 코드를 바이트 코드 형태로 변환하여 저장하는 JIT(Just-In-Time) 컴파일러를 사용한다면 Python 실행 속도를 개선할 수 있다.

PyPy는 Cython이나 Python에 비해 2배에서 3배 정도 성능향상을 보여주며 파일을 읽고 쓰기가 잦은 경우의 경우는 파일 입출력 속도의 문제로 느린 경향도 존재한다. 본 연구에서는 PyPy를 적용해 약 2.1배 정도 성능향상을 얻었다. 이에 관한 성능향상의 차이는 그림 41과 같다.

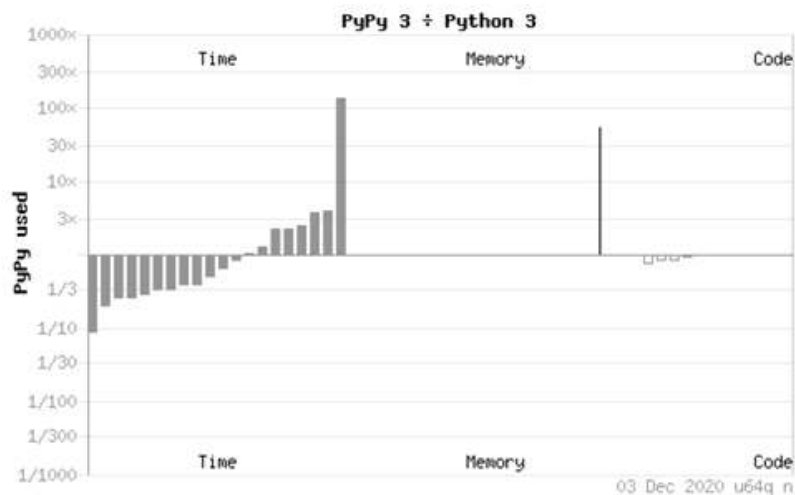


그림 41. Performance comparison graph between PyPy and Python

리스트 열거 기법은 소스 코드의 내용을 조금 수정한 기법으로 전통적으로 쓰는 for 반복문을 Python에서 제공하는 Lambda expression과 filter 함수 등을 활용해 더욱 간결하게 적을 수 있으며 for 반복문보다 성능이 더욱 빠르다고 할 수 있다. 이를 적용하기 전은 그림 42이고 적용한 후는 그림 43과 같다.

```
output = []
for element in large_list:
    if some_filter(element):
        output.append(element)
```

그림 42. Before applying list comprehension

```
output = list(filter(lambda x: some_filter(x), large_list))
```

그림 43. After applying list comprehension

문자열 병합 속도 최적화 기법은 Python에서 일반적으로 문자열을 합칠 때 Native 연결 방법이 사용되는데 이는 일차원 배열 공간에서 연결하는 방법으로 연결을 여러 번 반복하다 보면 문자열이 메모리에 많아져 연결하는 시간이 많이 소요된다. 이런 경우에는 문자열 전체를 하나의 리스트에 넣어 맨 마지막에 연결(join)하면 성능이 더욱 향상된다. 이를 적용하기 전은 그림 44이고 적용한 후는 그림 45와 같다.

```
out_str = ''
for num in xrange(loop_count):
    out_str += `num`
```

그림 44. Before applying string concatenation optimization

```
str_list = []
for num in xrange(loop_count):
    str_list.append(`num`)
return ''.join(str_list)
```

그림 45. After applying string concatenation optimization

Python에서는 기본적으로 하나의 코어만을 사용해 프로그램을 실행하는 단일 프로세싱을 사용한다. 멀티프로세싱 기법을 사용해 이를 보완하였다. Python에서도 동시성을 고려해 멀티스레드 방식이나 멀티프로세싱 기법을 사용할 수 있도록 라이브러리를 제공하고 있다. 이를 통해 Python의 성능을 향상하여 아래 그림 46과 같이 여러 개의 코어가 처리할 수 있도록 변경하였다.

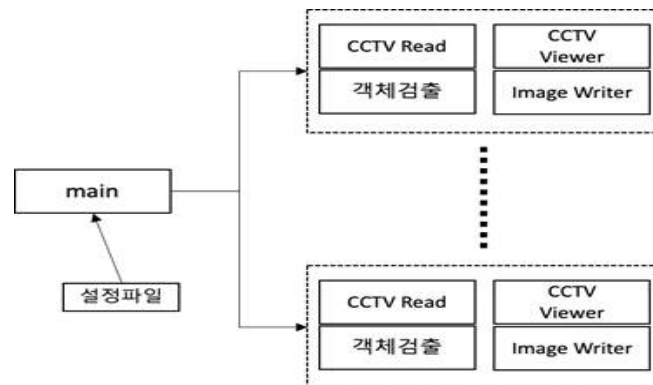


그림 46. CCTV multi-processing creating diagram

위 그림 46에서는 main 프로세스가 설정 파일을 읽어서 CCTV 개수와 IP 주소 등의 정보를 바탕으로 하나의 CCTV를 처리하기 위해 여러 개의 프로세스를 생성한 것을 표현한 것이다. 본 논문에서는 4개의 프로세스를 생성하였다.

CCTV Read 프로세스 IP 카메라에서 들어오는 영상을 전처리하고 전처리한 영상에 대해 객체 검출과 위험 판단 프로세스에게 이 영상을 넘기게 된다. 객체 검출과 위험 판단 이후에는 Image Writer 프로세스가 객체 검출 프로세스로부터 받은 영상을 통해 화면에 출력하고 추가로 영상을 디스크에 일자별 시간별로 저장할 수 있도록 하였다. 4개의 프로세스는 프로세스간 통신을 위해 같은 메모리를 사용해 성능을 향상했다.

## 4.5 시스템 구현 결과

본 관제시스템을 통해 CCTV 영상데이터를 실시간으로 모니터링함과 동시에 영상 내 객체의 움직임에 따라 객체를 검출하고 추적하여 상황을 볼 수 있도록 한 결과는 그림 47, 48과 같다.



그림 47. Real-time monitoring and object detection  
and tracking display 1



그림 48. Real-time monitoring and object detection  
and tracking display 2

그림 47은 4개의 CCTV 영상에 대해 왼쪽 상단의 경우 공장 외부, 오른쪽 상단은 출입문, 하단부는 실내 영상으로 구성되어 있다. 이 중 1개의 CCTV 영상이 크게 확대되어 표시한 것이 그림 48이다. 아래 그림 49는 과거의 CCTV 영상데이터를 저장하고 관리하며, 검색하고 위험 상황 발생에 대한 이력 조치사항을 검색 관리할 수 있도록 한 결과이다.

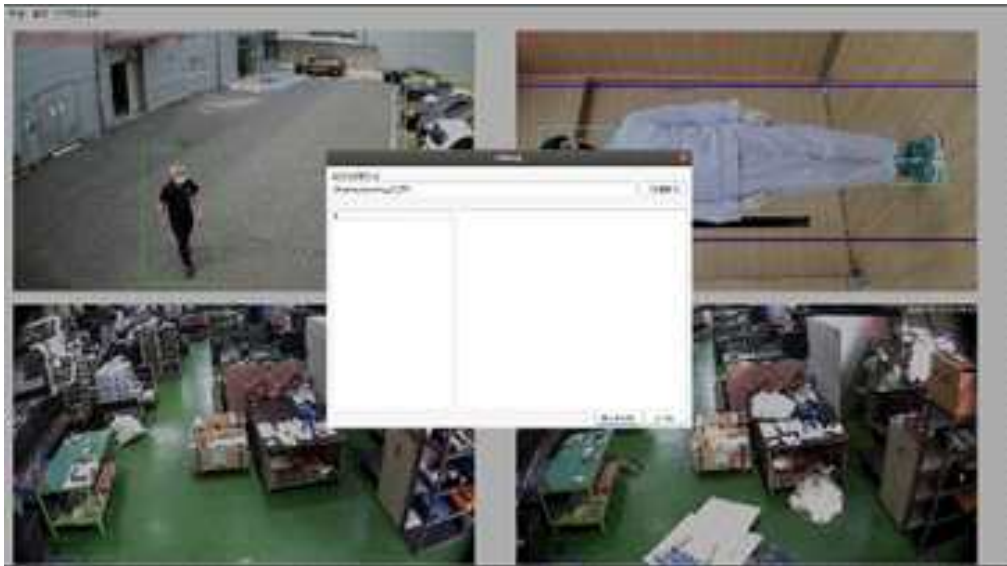


그림 49. Browse historical footage and view past dangerous events

아래 그림 50과 51은 객체 검출과 추적 결과를 바탕으로 위험 상황을 표시하기 위해 붉은색 박스로 출력하였으며 왼쪽 상단에 위험한 작업자 수를 카운트한 것이다.



그림 50. Display dangerous situations based on object detection results 1



그림 51. Display dangerous situations based on object detection results 2

아래 그림 52는 안전진단 상태를 오디오 신호로 생성하고 경광등을 표시하도록 결과를 촬영한 것이고, 그림 53과 54는 사업장과 사용자 관리를 위한 로그인 화면을 표시한 것이다.



그림 52. Generating audio signals for safety diagnostic status and output warning light indications



그림 53. Login screens for managing business and users 1

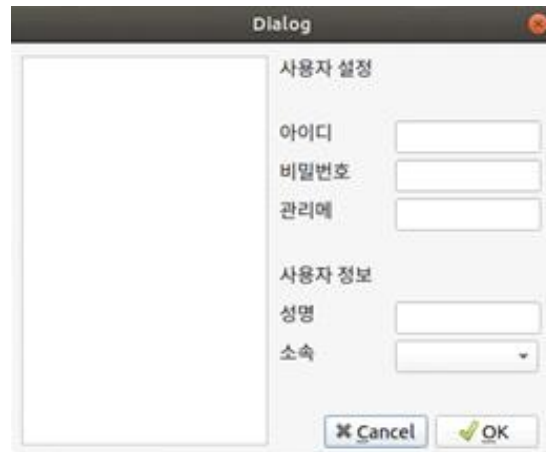


그림 54. Login screens for managing business and users 2

아래 그림 55와 56은 실증화 현장에 실시간 관제시스템을 구축한 촬영 사진이다.



그림 55. Established a [real-time](#) control system at the demonstration site 1



그림 56. Established a **real-time** control system at the demonstration site 2

## 4.6 고찰

본 논문에서는 실증 사업장에서 위험 상황을 판정하고 시나리오를 구성하여 CCTV 영상을 기반으로 한 관제시스템을 구축하였다. 객체 검출 모델은 YOLOv4를 사용해 IoU가 0.25인 상태에서 검출 정확도가 위험 구역 검출, 안전 장구와 위생 장구의 경우 100%의 검출 성능을 보였으며 실외 환경의 경우 검출 정확도가 65.61로 떨어지는 결과를 보였다. 이는 실외 환경에서 CCTV 설치 환경의 문제, 실외 환경만의 조도, 객체의 크기 등 실내 환경에 비해 제약 사항이 크기 때문이다. 실외 환경이 실내 환경과 다른 환경이고 이에 따른 추가적인 전처리 과정이 필요하다. 아울러 모델이 실내 환경 위주의 시나리오로 구성되어 있다는 점에서 모델 학습이 편향됐음을 알 수 있다.

또한 IoU가 0.25인 상태의 경우, 충돌 감지에서 CCTV가 설치된 환경에 따라 잘못된 검출 결과를 보일 수도 있다. CCTV가 비교적 낮은 각도에 설치되어 있는 경우 IoU 값을 작게 한다면 충돌이 일어나지 않은 상

황에서도 충돌이 일어났다고 판단할 수 있다. 반대의 상황에서도 충분히 잘못된 검출 결과를 보일 수 있어 객체의 거리, IoU 값을 실험으로 정확히 임곗값을 산출할 필요가 있어 보인다.

관제시스템을 구현 과정에서 안전관리 담당자, 작업자가 보기 쉬운 사용자 인터페이스 환경이 충분한지 조사와 검증을 통해 실증 사업장에서 활용하기 편리하도록 개선해야 할 점도 향후 연구 과제로 남겨 두고자 한다.

## V. 결론

본 논문에서는 실증 사업장 3곳을 선정하여 위험 상황을 판정하기 위한 시나리오와 위험 상황이 발생하기 전인 안전 규칙 위반 상황 시나리오도 함께 정립하였다. 이를 통한 도출된 시나리오를 통해 CCTV 영상을 수집하였으며 영상이나 영상 프레임이 부족한 시나리오나 객체에 대한 추가 수집도 함께 진행하여 모델의 편향을 극복하고자 하였다.

객체 검출 모델은 YOLOv4를 문헌 연구와 실험을 통해 선정하였으며 객체 검출 결과는 99%, 시나리오에 대한 상황인식 결과는 65.61%에서 100%의 결과를 보였다. 하지만 이러한 결과는 IoU 값이 0.25로 제한된 설정이며 수집한 데이터에서 K-fold 교차 검증 방법을 활용한 값이다. IoU가 0.25로 제한된 값이기 때문에 실내나 실외 환경, 다른 사업장의 경우는 임곗값을 별도로 산출해야 한다.

이러한 점을 보완하고자 IoU 임곗값뿐만이 아닌 영상에서 객체와의 거리 계산, 이전 프레임과 현재 프레임에 등장한 객체가 동일한 객체인지를 판단하기 위한 유사도 검사 등이 함께 기능으로 추가하였다.

관제시스템 설계 과정에서는 기능별 추출과 IP 카메라를 별도로 설치해 영상 모니터링과 대시보드 기능을 구현하였다. 관제시스템이 객체 검출과 위험판정, 규칙 위반 판정, 이를 통한 알람 기능까지 모두 수행하려면 기존의 설계한 관제시스템에는 처리 속도나 처리량에 따른 한계가 존재한다. 이를 방지하고자 Python 코드 실행시간 최적화 기법, 리스트 열거, 문자열 병합 속도 최적화, 멀티프로세싱 기법을 구현해 이를 개선하였다.

이를 토대로 IoT 센서, 네트워크 구축 등을 통해 산업안전 재해 방지를 위한 시스템 연구에 활용될 수 있을 것이며 관제시스템이 더욱 강건해지기 위한 추가 데이터 수집을 위한 플랫폼으로도 활용할 수 있을 것이다.

## 참고문헌

- [1] Korea Ministry of Employment and Labor. (2023, Jun. 20). *Occupational hazard factors (SIFs) resources*. [Online]. Available: [https://www.moel.go.kr/local/seoulbukbu/news/notice/noticeView.do?bbs\\_seq=20230600978](https://www.moel.go.kr/local/seoulbukbu/news/notice/noticeView.do?bbs_seq=20230600978)
- [2] Korea Ministry of Employment and Labor. (2023, Mar. 02). *The state of occupational injuries in 2022*. [Online]. Available: [https://www.moel.go.kr/policy/policydata/view.do?bbs\\_seq=20230300058](https://www.moel.go.kr/policy/policydata/view.do?bbs_seq=20230300058)
- [3] M. Everingham, L. VanGool, C. K. I. Williams, J. Winn, and A. Zisserman. “The PASCAL visual object classes challenge 2007(VOC2007) results,” Mar. 2008. Distributed by University of Oxford. [Online]. Available: <http://www.pascal-network.org/challenges/VOC/voc2007/works hop/index.html>
- [4] M. Everingham, L. VanGool, C. K. I. Williams, J. Winn, and A. Zisserman. “The PASCAL visual object classes challenge 2012(VOC2012) results,” Nov. 2013. Distributed by University of Oxford. [Online]. Available: <http://www.pascal-network.org/challenges/VOC/voc2012/works hop/index.html>
- [5] T. Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, “Microsoft COCO:

- common objects in context,” *in proceeding of computer Vision–ECCV 2014: 13<sup>th</sup> European conference*, pp. 740–755, 2014.
- [6] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R–CNN: towards real–time object detection with region proposal networks,” *in proceedings of advances in neural information processing systems 28*, 2015.
- [7] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C–Y. Fu, and A. C. Berg, “SSD: single shot multibox detector,” *in Computer Vision – ECCV 2016: 14<sup>th</sup> European Conference*, Amsterdam, Netherlands, pp. 21–37, Oct. 2016.
- [8] S. Zaidi, M. S. Ansari, A. Aslam, N. Kanwal, M. Asghar, and B. Lee, “A survey of modern deep learning based object detection models,” *Digit. signal process*, 2022.
- [9] J. Dai, Y. Li, K. He, and J. Sun, “R–FCN: object detection via region–based fully convolutional networks,” *in Proceedings of advances in neural information processing systems 29*, 2016.
- [10] C. Y. Wang, H. Y. M. Liao, Y. H. Wu, P. Y. Chen, J. W. Hsieh, and I. H. Yeh, “CSPNet: a new backbone that can enhance learning capability of CNN,” *In proceedings of the IEEE/CVF conference on computer vision and pattern recognition(CVPR) workshops, Virtual*, 2020.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, “Spatial pyramid pooling in deep convolutional networks for visual recognition,” *IEEE trans. pattern anal. mach.*

- intell. (TPAMI)*, 2015.
- [12] A. Bochkovskiy, C. Y. Wang, and H. Y. M. Liao, “YOLOv4: optimal speed and accuracy of object detection,” *arXiv:2004.10934*, 2020.
  - [13] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, “Path aggregation network for instance segmentation,” *In proceedings of the IEEE conference on computer vision and pattern Recognition (CVPR)*, 2018.
  - [14] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. C. Chen, “MobileNetV2: inverted residuals and linear bottlenecks,” *In proceedings of the IEEE conference on computer vision and pattern recognition*, 2018.
  - [15] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” *in proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 580–587, 2014.
  - [16] W. Lina, S. Hailong, Z. Cheng, M. Zuqiang, L. Huiwu, L. Xichun, and Y. T. Yuan, “Hyperspectral image classification using wavelet transform-based smooth ordering,” *Int. j. wavelets multiresolution inf. process*, 2019.
  - [17] E. Guariglia, “Primality, fractality, and image analysis,” *Entropy*, 2019.
  - [18] R. Girshick, “Fast R-CNN,” *in Proceedings of the IEEE international conference on computer vision*, pp.

- 1440–1448, 2015.
- [19] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: unified, real-time object detection,” *in proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 779–788, 2016.
  - [20] K. Duan, S. Bai, L. Xie, H. Qi, Q. Huang, and Q. Tian, “CenterNet: keypoint triplets for object detection,” *In proceedings of the IEEE/CVF international conference on computer vision*, 2019.
  - [21] S. Kim, J. Lee, and C. Kang, “Analysis of industrial accidents causing through jamming or crushing accidental deaths in the manufacturing industry in south korea:Focus on non-routine work on machinery,” *Saf.Sci*, 133, 104998, 2021.
  - [22] J. Redmon, and A. Farhadi, “YOLO9000: better, faster, stronger,” *in Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 7263–7271, 2017.
  - [23] J. Redmon, and A. Farhadi, “YOLOv3: an incremental improvement,” *arXiv:1804.02767*, 2018.
  - [24] R. Padilla, S. L. Netto, and E. A. B. da Silva, “A survey on performance metrics for object-detection algorithms,” *in 2020 international conference on systems, signals and image processing (IWSSIP)*, pp. 237–242, 2020.
  - [25] OpenCV, “Computer vision annotation tool(CVAT),” Github

- .com. <https://github.com/opencv/cvat> (accessed Aug. 20, 2023).
- [26] AlexeyAB, “Yolo\_mark,” Github.com. [https://github.com/AlexeyAB/Yolo\\_mark](https://github.com/AlexeyAB/Yolo_mark) (accessed Apr. 24, 2019).
- [27] Darkpgmr, “DarkLabel,” Github.com. <https://github.com/darkpgmr/DarkLabel> (accessed Sep. 4, 2021).
- [28] T. Lee, S. Kim, C. Hwang, and H. Jung, “Worker collision safety management system using object detection,” *Journal of the korea institute of information and communication engineering*, vol. 26, no. 9, pp. 1259–1265, Sep. 2022.
- [29] T. Lee, Y. Kim, and H. Jung, “Dataset construction and model learning for manufacturing worker safety management,” *Journal of the korea institute of information and communication engineering*, vol. 25, no. 7, pp. 890–895, Jul. 2021.
- [30] T. J. Lee, K. S. Woo, P. Y. Kim, and H. K. Jung, “Design and implementation of industrial accident detection model based on YOLOv4,” *Applied Sciences*, Vol. 13, pp 1–19. 2023.
- [31] H. Ahn, S. Son, K. Park, S. Kim, “Cost assessment model for sustainable health and safety management of high-rise residential buildings in Korea,” *J. asian archit. build. Eng.* 2021.
- [32] W. Y. Kim, H. H. Cho, “Unions, health and safety committees, and workplace accidents in the korean

- manufacturing sector,” *Saf. health work* 2016.
- [33] F. A. S Snachez, G. I. C Pelaez, and J. C. Alis, “Occupation safety and health in construction: A review of applications and trends,” *Ind. Health* 2017, pp. 210–218. 2017.
- [34] M. A. Friend and J. P. Kohn, “Introduction to occupational safety and Health,” *In fundamentals of occupational safety and health 8th*, pp. 1–24, 2023.
- [35] T. J. Lee, Y. J. Kim and H. K. Jung, “Dataset construction and model learning for manufacturing worker safety management,” *J. korea inst. inf. comm. eng.*, 2021.
- [36] H. Baek, D. H. Kim, and Y. Jeon, “A study on disclosure items of safety and health management system for major injury prevention,” *Crisisonomy* 2022.
- [37] J. Min, Y. Kim, S. Lee, T. W Jang, I. Kim, and J. Song, “The fourth industrial revolution and its impact on occupational health and safety, workers compensation and labor conditions,” *Saf. health work* 2019.
- [38] S. Y. Kang, S Min, D. Won, Y. J. Kang, and S. Kim, S, “Suggestion of an improved evaluation method of construction companies industrial accident prevention activities in South Korea,” *Int. J. Environ. Res. Public Health* 2021.
- [39] S. Y. Kang, S. Min, W. S. Kim, Y. J. Kang, and S. Kim, “Types and characteristics of fatal accidents caused by multiple processes in workplace: based on actual cases in

- South Korea,” *Int. J. environ. res. public health* 2022.
- [40] S. H. Hwang, S. Kang, and Y. Shin, “Study on the risk assessment of fall accidents at construction sites through the analysis of accident cases,” *J. Korea acad. Ind. coop. soc.* 2022.
- [41] J. Moon, “The effect of subcontractor status on occupational injury and disease incidence: A cross-sectional study using the 9th occupational safety and health company survey,” *Int. arch. occup. environ. health* 2022.
- [42] B. Y. Jeong, “Characteristics of occupational accidents in the manufacturing industry of south korea,” *Int. J. Ind. Ergon*, Vol. 20, pp. 301–306, 1997.
- [43] J. Kwon, “A study on measures to promote industrial accident prevention in workplaces with less than 50 employees,” *Master’ s thesis, Korea university*, Jul. 2009.
- [44] J. S. Ro, J. H. Leigh, I. Jeon, and M. S. Bang, “Trends in burden of work-related upper limb amputation in South Korea,” , *A nationwide retrospective cohort study*, 2009
- [45] S. C. Ko, T. S. Eum, and C. G. Song, “Development of disaster safety program for small-sized workplaces and its introduction to occupational safety&health act: focusing on fire,” *Explosion and chemical spill. J. Converg. Inf. Technol*, 2021.
- [46] S. D. Choi, L. Guo, J. Kim, and S. Xiong, “Comparison of fatal occupational injuries in construction industry in the United States,” *South Korea, and China. Int. J. Ind. Ergon*,

2019.

- [47] J. Choi, S. Lim, S. Park, H. Roh, H. Jin, and C. Lee, “The serious accidents punishment act of South Korea and its impact on the shipping industry: toward sustainability,” *Sustainability*, 2022.
- [48] J. Hwang, “Criminal justice in a time of ecological crisis: can the serious accidents punishment act in korea be enforced to punish ecocide,” *Int. J. Crim. Justice* 2022.
- [49] L. Cai and Y. Zhu, “The challenges of data quality and data quality assessment in the big data era,” *Data sci. j.* 2015.
- [50] S. Picard, C. Chapdelaine, C. Cappi, L. Gardes, E. Jenn, B. Lefevre, and T. Soumarmon, “Ensuring dataset quality for machine learning certification,” *In proceedings of the 2020 IEEE international symposium on software reliability engineering workshops (ISSRE)*, 2020.
- [51] Ministry of Trade, Industry and Energy, “Safety accident prevention technology development aiming for specific industrial needs utilizing AI-based image processing technology,” Result report 2022.
- [52] S. Vicente, J. Carreira, L. Agapito, and J. Batista, “Reconstructing PASCAL VOC,” *In proceedings of the IEEE conference on computer vision and pattern Recognition (CVPR)*, 2014.
- [53] H. S. Chae, “Comparison and analysis of object detection models for detecting dangerous situation of workers in industrial sites,” *Master’ s thesis, Paichai university*, 2022.

# A Study on CCTV-based Occupational Safety Control System

Sooyoung Ryoo

*Department of Computer Engineering,  
Graduate School of Paichai University  
Daejeon, Korea*

(supervised by Professor Hoekyung Jung)

Industrial accidents are still occurring even after the implementation of the Fatal Accident Punishment Act, and various efforts are being made by the government and industry to reduce them. In particular, according to the 2023 Serious Injury and Fatality (SIF) data published by the Ministry of Employment, Labor and Welfare and the Korea Occupational Safety and Health Agency, the construction industry and manufacturing industry accounted for 4,432 accidental deaths in the last six years, and among them, 74 classification types and 29 high-risk tasks were reported as causing accidents in the manufacturing industry. These included poor loading conditions, power outages, and being caught in moving equipment.

In order to prevent such industrial accidents, risk reduction measures are strongly required at workplaces and safety management personnel are legislated to be in charge. However, it is not easy for them to analyze all risk factors. To compensate

for this, CCTV cameras installed in workplaces are used for management and supervision, and the data is stored and used to detect abnormal behavior. The Korea Intelligent Information Society Agency also provides various data related to occupational safety. For abnormal behavior detection, we use object detection and key point extraction, which is human joint data. Many models such as SSD and YOLO are used for object detection, and models such as OpenPose, HRNet, and AlphaPose are used for key point extraction.

In this paper, we selected scenarios and extracted objects from CCTV images acquired from three business sites. The objects include workers, dangerous areas, safety equipment such as hard hats and safety shoes, and hygiene equipment, and the scenarios include loading materials, not wearing safety equipment or hygiene equipment, and collision of moving objects such as forklifts, which are dangerous situations or violations of rules common to all three workplaces.

To recognize these scenarios and objects, an object detection model is essential, and YOLOv4 was finally selected through literature research and experiments. Using the hyperparameters of YOLOv4, the object detection performance was 99%, and the situation recognition of the scenario showed the lowest value of 65.61% and the highest value of 100% in the process of detecting workers in the outdoor environment.

Then, to build the control system, the functions were extracted and designed by modules. In order for the control system to perform both object detection and risk determination or rule violation alarm, there is a problem in terms of speed with the existing designed control system, and to prevent this, Python code execution time optimization techniques, the use of list enumeration, string merge speed optimization, and multiprocessing techniques were used.

Based on the research conducted in this paper, a system for preventing industrial safety accidents is constructed by utilizing various sensors and networks. Furthermore, the system can be applied not only to prevent industrial safety accidents but also to enhance the robustness of such systems through the collection of large-scale data. It is also anticipated that the system can be utilized in the development of performance improvement through keypoint extraction of workers.

## 감사의 글(Acknowledgement)

子曰: “學而時習之, 不亦說乎

有朋自遠方來, 不亦樂乎

人不知而不慍, 不亦君子乎”

자왈: “배우고 때때로 그것을 익히면 또한 기쁘지 않은가?

친구가 먼 곳에서 찾아오면 또한 즐겁지 않은가?

남이 알아주지 않아도 성내지 않는다면 또한 군자답지

않은가?”

배움에는 끝이 없다는 옛 성현의 말씀처럼, 나이가 더해져도 계속해서 배움의 길을 걷고자 합니다.

1만겹 세월의 연을 통해 스승과 제자로 인연을 맺게 되어, 부족한 저에게 변함없는 관심과 애정으로 지도해 주신 정회경 교수님께 깊은 감사와 존경을 표합니다.

박사과정 동안 함께한 배재대 MIE(Multimedia Information Engineering) 박사회 회원분들과 많은 도움을 주신 MIE 연구실 후배님들께도 감사의 인사를 드립니다.

또한 이 어려운 여정을 함께한 박사과정 동기들에게도 깊은 감사를 전하며, 함께한 순간들이 풍요로운 추억으로 남을 것임을 기대합니다.

마지막으로 자식들을 걱정하며 항상 기도해주신 어머니와 친아들처럼 따뜻한 보살핌을 주시는 장인어른과 장모님, 가족을 위해 무한한 사랑을 실

천하고 헌신해 주신 아내와 사랑스러운 세 아들 진우, 진혁, 진에게 사랑  
과 감사의 마음을 전합니다.

이 모든 것이 저의 학문의 성취에 큰 힘이 되었습니다.

다시 한 번 감사의 말씀을 드립니다.

2023년 12월

류 수 영 드림